BACK TO METAL

WHY THIS BOOK EXISTS

YOU ARE ALLOWED TO RUN YOUR OWN COMPUTERS

A generation of engineers has now been trained to believe that owning a server is a kind of professional failure. It is not. It is a trade, and like every trade it has terms: you choose dedicated capacity, rented or owned in colocation, and keep more of the budget as your workload grows. The cloud-ops team transitions to on-prem ops with the same planned hours, while remote hands handles physical interventions. For a company with steady demand and a substantial cloud bill, both routes deserve a serious comparison.

What has been missing is not the argument. It is a plan short enough to finish. Repatriation is written about as a decision and executed as a crisis, which is why so many attempts stall halfway, with two platforms live and nobody able to say whether it is going well. This is the other thing: 20 Moves, in five stages, each one a single job with a stated cutover, a stated risk and a rollback that works. An earlier edition of this book had a hundred and twenty-two of them and took four years. Nobody has four years.

It is honest about the parts that do not pay. One whole Move ends by telling you to keep paying somebody else, because a content network, denial-of-service scrubbing and outbound email are three things you will not beat on your own, and a book that pretends otherwise is selling something. Another gives you permission to read three Moves, do the arithmetic, and stop. The point was never to own everything. It was to own the parts where ownership is cheaper, faster and yours.

Which is why the whole thing is **open source**. Every Move, the typesetter that turns them into this book, and the site that publishes them are yours: the text under Creative Commons, the software under the MIT licence, all of it at github.com/OneUptime/back-to-metal. Correct it, extend it, add the services your own estate actually runs. Infrastructure knowledge this practical should not sit behind a consulting invoice.

BACK TO METAL

How a company leaves the cloud, in twenty moves

Nawaz Dhandala

BY THE MAKERS OF ONEUPTIME · ONEUPTIME.COM

HACKERBAY · HACKERBAY.IO

BACK TO METAL

How a company leaves the cloud, in twenty moves

Published by HackerBay, HackerBay.io

First edition, 2026. Version 6.3.1.

Every runbook in this book changes production infrastructure, and several of them move data that cannot be recreated. Read the Rollback section before the first step of any Move, take a backup you have restored at least once, and rehearse against a copy. Prices, service names and product behaviour are those the author observed while writing and will drift; the figures in each Move are illustrative of the shape of a saving, not a quotation. Nothing here is legal, financial, security or compliance advice. You remain responsible for your own systems, your own data and your own obligations to the people whose data it is.

Disclosure: the author founded OneUptime, which this book recommends for alerting, on-call, incidents and status pages. It is recommended because it is Apache-2.0 licensed, self-hostable, and does in one place what that Move would otherwise ask you to assemble from four separate tools. The alternatives named beside it are real ones, and a reader who prefers them loses nothing else in this book.

Set in Archivo, drawn by Omnibus-Type, and JetBrains Mono, drawn by Philipp Nurullin and Konstantin Bulenkov. Both are licensed under the SIL Open Font License.

backtometal.hackerbay.io

A SHORT NOTE FIRST

MANY MOVES, NOT ONE MIGRATION

The reason leaving the cloud fails is almost never that the technology did not work. It is that it was attempted as one decision, executed as one project, and abandoned somewhere in the middle with two platforms running and nobody able to say whether it was going well.

This book takes the opposite shape. It is 20 Moves, in five stages. Each is one job with a stated cutover, a stated risk and a rollback that has been thought about. Each can be done on a Tuesday and undone on a Wednesday. You can stop after any of them and still be in a coherent place, which is the only property that matters in a project measured in quarters.

It is deliberately small. The first edition had a hundred and twenty-two Moves and, computed from its own files, 975 person-days and four and a half years. That was correct and useless. This is 107 person-days and about 44 weeks for two people, most of it hardware lead time rather than work. Hiring does not shorten it.

They are ordered so that a reader who starts at Move 01 and works forward has, by construction, met every prerequisite of the Move in front of them. That is not a stylistic choice; the build refuses to compile a book where it does not hold.

Every Move carries all three clouds. The job is the same whether you are leaving AWS, Google Cloud or Azure — what differs is the extraction, so each Move opens with three lines naming the real service on each provider and the one thing that is different there.

Move 04 concludes that you should keep paying somebody else for three things. A content delivery network, scrubbing capacity at the edge and outbound email are businesses other people run better than you will, and each is cheap next to what it replaces. The aim was never to own everything. And Move 03 gives you permission to stop: if the arithmetic does not clear a third, three Moves and a week is the whole cost of finding out.

Rehearse everything, and read the Rollback before the first step of every Move — including the ones that look like nothing.

EVERY MOVE, WITHOUT EXCEPTION

HOW A MOVE WORKS

1 **THE RING**
The cutover: how many minutes of user-visible downtime this Move costs, against a 60-minute dial. An empty ring means none, and most rings are empty.

2 **THE BARS**
Risk, meaning blast radius if it goes wrong rather than how fiddly it is. Three bars and you do not run it alone.

3 **THE PANEL**
Everything that must already be true before step one. Hardware, software with versions, and the access you will be asked for.

4 **WHY THIS WORKS**
What the managed service actually was, and why the replacement holds. Read it once; you will not need it again.

5 **THE STRIP**
What it cost, what it costs now, the saving, the downtime and the person-days. Illustrative of a shape, not a quotation for your account.

6 **THE ROLLBACK**
On the facing page, at the foot, in colour. It names the point of no return. Read it before step one, not after step five.

BEFORE THE TWENTY MOVES, THE QUESTION UNDERNEATH THEM

WHY LEAVE AT ALL

Colocation and rented dedicated servers put more of your infrastructure budget into capacity you can use. Rent the machines to preserve cash, or own them in a colocation facility for lower running costs and control over the hardware. In this reference model, colocation saves $4,886,320 over five years, with the same operations hours as the cloud. Your team brings its skills; the provider supplies the building and physical support.

WE DID THIS, AND HERE IS WHAT ACTUALLY HAPPENED

730 days
at 99.993 per cent measured availability, through a period that included a region-wide outage at the cloud we had left

19%
lower latency on the same software, from local NVMe and the absence of neighbours. We bought no faster code to get it

2
hardware interventions in twenty-four months, both disks, both handled by remote hands with a mean response of twenty-seven minutes

0
people hired. The toil moved; it did not multiply. We measured it at about fourteen engineer-hours a month across two sites

Our own fleet and our own measurement, not an independent audit, and a bigger estate than this book is sized for. The rest of the figures here stand on outside sources and on arithmetic you can repeat.

1 THE BILL STOPS BEING A PERCENTAGE OF YOUR GROWTH

This is the one that matters and the rest are consequences of it. A cloud bill is a toll on activity: more users, more bytes, more bill, for ever, at a margin somebody else sets and can change. A rack or a dedicated-server contract buys capacity at a predictable price. While your workload fits that capacity and your power and traffic allowances, more users need not mean more infrastructure rent. Add machines when demand calls for them. Over five years the difference here is $4,886,320, and the machines are still working at the end of it.

> CHECK IT
> Take last quarter's invoices and plot the total against your own usage metric. If the line is flat you have nothing to gain here. If it tracks your growth, that slope is what you are buying out of — Move 01.

2 EGRESS STOPS BEING A TAX ON YOUR OWN TRAFFIC

Moving your own data to your own users is where the margin lives, and it is the line that ends most of these arguments: $7,980 a month to send a hundred terabytes out. Transit at a facility is bought by the megabit and costs a small fraction of that. Nothing you build gets faster and no code changes; the line simply stops being there.

> CHECK IT
> Find the data-transfer line on last month's bill. Divide it by your egress in terabytes. If it is anywhere near ninety dollars a terabyte, that line is nearly pure margin — Move 01, then Move 18.

3 THE MACHINES ARE YOURS, SO THE PERFORMANCE IS YOURS

A vCPU is half a core somebody else is also using. Local NVMe is not a network service with a queue in front of it and a token bucket on top. The gain is not theoretical and it is not small: on identical software we measured nineteen per cent, and the noisy-neighbour tail — the ninety-ninth percentile that wakes people — improves more than the mean does.

> CHECK IT
> Compare your ninety-ninth percentile against your median for a week. If the gap is wide and unexplained by your own code, some of it is not yours — Move 14 measures it on real hardware before anything moves.

4 NOBODY DEPRECATES YOUR HARDWARE

A managed service is somebody else's roadmap running inside your product. Instance families are retired, versions go end-of-life on a date you did not pick, a control plane upgrades on its own release channel, and the longest you can hold it is ninety days. A machine you own runs the version you chose until you choose another one, and the migration happens when you have time rather than when the notice arrives.

> CHECK IT
> Count the forced upgrades and deprecation notices you have absorbed in the last two years, and what each cost in engineer-days. That is a recurring bill nobody invoices you for.

5 YOUR TEAM RUNS THE PLATFORM; REMOTE HANDS LOOK AFTER THE RACK

Your cloud-ops engineers already deploy, monitor, patch and recover services. Those skills move with the workload. Colocation remote hands carry out disk swaps, cabling and power cycles under your runbooks; a rented-metal provider maintains its hardware under the support agreement. The reference model keeps the same team and monthly operations hours across all three options.

> CHECK IT
> Agree the physical tasks, coverage and response times with the provider, then rehearse an incident together. Measure your team's hours before and after the move — Moves 07 and 19.

TWO ROUTES, ONE OPERATIONS TEAM

CHOOSE HOW YOU MOVE

Both routes can lower the bill. Choose the purchasing and support arrangements that fit your cash flow and the team you already have.

1. **RENT FIRST, OR OWN IN COLOCATION**
Rented metal lets you move without buying the servers upfront. Colocation lets you own the hardware and spread its purchase cost over years of use. Compare both cash flows in Move 03 and choose the route in Move 07.

2. **A TIMETABLE THAT FITS THE ROUTE**
Available rented servers can avoid the hardware purchase and facility setup lead times. The colocation plan includes ordering machines, signing space and waiting for circuits: 107 person-days of work across 44 weeks. Both routes keep the migration rehearsals and rollback windows.

3. **THE SAME TEAM, THE SAME OPERATIONS HOURS**
Cloud ops transitions into on-prem ops. Remote hands handles the physical interventions; your engineers keep responsibility for the platform and application. The model budgets 160 hours a month in every option, with remote hands in the facility bill. Migration work is counted separately.

4. **THREE THINGS THAT NEVER COME HOME**
A content delivery network, outbound mail deliverability and scrubbing at the edge are businesses other people run better than you will. Move 04 tells you to keep paying for all three. These and the services retained in later Moves total $9,180 a month in the reference model.

WHEN TO STAY EXACTLY WHERE YOU ARE

Your load is spiky or seasonal and genuinely scales to near nothing between the spikes. Elasticity is the one thing a rack cannot do, and paying for the peak all month is how owning becomes the expensive answer.

You lean hard on managed services that have no real equivalent - a serverless database, a hosted stream, a workflow engine - and replacing them is a rewrite rather than a migration. Move 02 will tell you this in an afternoon.

You have no team to own the platform and no managed operations partner. Remote hands covers physical work; application recovery and platform decisions still need an owner. An existing cloud-ops team can transition into that role.

Your effective cloud bill is too small to cover migration and the capacity you need. Price rented metal as well as a colocation rack: renting can suit a smaller estate. Move 03 compares your own quotes and hours, so the decision rests on your workload rather than a spending threshold borrowed from another company.

If none of those four is true and the arithmetic in Move 03 clears, the rest of this book is twenty jobs that get you there, each one done on a Tuesday and undone on a Wednesday.

THE REFERENCE BUILD

Every Move in this book is written against one cluster, so that a runbook can name a real thing rather than a category: one site, 16 nodes and 2 on the shelf, 32 cores and 256 GB a node, 25 GbE to the switch. 16 because that is what the bill buys and a little over, and because at this size losing one costs six per cent of the fleet rather than a third; the spares because a dead board is a return authorisation and three weeks. Scale the numbers; do not scale away the redundancy.

COMPUTE

- [ ] Five 1U dual-socket servers, and a sixth on the shelf
- [ ] 32 physical cores a node, current-generation EPYC or Xeon
- [ ] 256 GB ECC memory a node, all channels populated
- [ ] Redundant hot-swap power supplies, each fed from a different strip
- [ ] A management controller on every node - Redfish, not just a web console

STORAGE

- [ ] Four NVMe drives a node, mixed-use endurance, 3.84 TB each
- [ ] Enterprise drives with power-loss protection - not consumer SSDs
- [ ] A separate boot pair, mirrored
- [ ] One spare drive of each size, on site, unopened
- [ ] A backup target at a provider you are not otherwise using

NETWORK AND SPACE

- [ ] Two 25 GbE switches, stacked or in a link-aggregation pair
- [ ] Two uplinks a node, one to each switch
- [ ] A separate 1 GbE management switch, not reachable from the internet
- [ ] A full rack in a carrier-neutral facility, A and B power
- [ ] Facility transit for go-live, and a second upstream ordered alongside it
- [ ] An out-of-band line that works when the main path does not

THE SOFTWARE STACK

- [ ] Talos Linux on every node - no shell, no package manager, no drift
- [ ] Kubernetes, on a version at least one minor behind the newest
- [ ] Cilium for the cluster network, with kube-proxy replaced
- [ ] Rook, and the Ceph cluster it manages
- [ ] CloudNativePG, on local NVMe rather than on Ceph
- [ ] cert-manager and Envoy Gateway
- [ ] Argo CD, with the cluster state in Git
- [ ] Prometheus and Loki for the stores, OneUptime for what wakes somebody
- [ ] Velero, and a restore you have actually performed

ACCESS

- [ ] An identity provider that is not the cluster
- [ ] Break-glass credentials in a safe, on paper
- [ ] A jump host or a mesh - never a public API server
- [ ] Hardware tokens for anyone who can delete a cluster
- [ ] A status page that is not hosted on the thing it reports on
- [ ] The domain registrar account locked, with recovery codes printed

ON THE LAPTOP

- [ ] kubectl
- [ ] helm
- [ ] talosctl
- [ ] k9s
- [ ] argocd
- [ ] velero
- [ ] psql and pg_dump
- [ ] barman-cloud
- [ ] redis-cli
- [ ] rclone
- [ ] the cloud CLI, still logged in
- [ ] dig and mtr
- [ ] openssl
- [ ] ipmitool
- [ ] the runbook, printed

THE ON-RAMP

Almost nobody should sign a facility contract before they have run this stack once. A cluster of 3 refurbished machines with 12 cores and 64 GB each, on a managed switch under a desk, will run every Move in Stage 3 unchanged and most of Stage 4 besides. It costs about $3,430 once and about $30 a month in electricity, it saves nothing whatsoever, and it is the cheapest way to find out whether the rest of this book is for you.

WHY THREE, AND WHY SECOND-HAND

Three because a control plane needs three members to lose one and keep going, and everything you will learn about quorum, fencing and upgrades needs somewhere to go wrong. Second-hand because a machine two generations old runs Kubernetes exactly as well as a new one at roughly a tenth of the price, and because you want to be willing to break it.

WHAT IT GENUINELY PROVES

The whole software estate. Talos, the control plane, Cilium with kube-proxy replaced, Rook and Ceph, the registry, secrets, the observability stack, and a Postgres cutover rehearsed end to end against a copy. If a Move works here it will work on the metal; the difference is scale, not shape.

WHAT IT CANNOT TEACH YOU

Every word of Stage 2. A homelab has one power feed, one switch, no cross-connect, no remote hands and nobody to escalate to at three in the morning. It cannot show you what a colocation contract is for, or what happens when the A feed goes away and the B feed was never tested. The building is a different discipline, which is why it has a stage of its own.

WHEN TO STOP USING IT

Never. The on-ramp becomes the non-production cluster the rest of the book keeps asking for — the one you restore etcd onto, rehearse an upgrade on, and point a load generator at. Every estate needs a cluster it is allowed to destroy, and this is the cheapest one you will ever own.

THE ONE PART TO BUY NEW

The drives. A consumer SSD without power-loss protection will corrupt etcd the first time the power flickers, and it will do it in a way that looks like a Kubernetes bug for a day and a half. Buy small enterprise NVMe with the protection, second-hand if you like, and put the money you saved on the chassis into the disks.

IF THREE MACHINES AT HOME IS NOT POSSIBLE

Rent three suitable dedicated hosts available on monthly terms and run the same experiment. It costs more than electricity and less than being wrong, it gives you real addresses and a real network, and you can end the rental under the provider's notice terms. What it will not give you is a machine you can physically pull a disk out of.

THE WHOLE SHOPPING LIST

- [ ] Three refurbished mini PCs or small-form-factor desktops
- [ ] 64 GB in each - memory is what runs out first, not cores
- [ ] Two NVMe drives a node: one to boot, one for data
- [ ] A managed switch with VLANs and a spare port for out-of-band
- [ ] A small uninterruptible power supply, mostly to survive the brownouts that corrupt etcd
- [ ] A domain you own, for real certificates rather than self-signed ones

THE RULES FOR LEAVING THE CLOUD

1 **Move the cheapest thing first.** Not the most expensive. The first Move exists to prove the platform and the team, and it should be something whose failure is a shrug. You are buying evidence, and evidence is worth more early than savings are.

2 **Data moves last and leaves last.** Every stateless thing can be cut over and cut back in minutes. State cannot. Move compute first, run it against the cloud database across the link, and only then move the database - by which point you have already learned the things you would otherwise learn during a data migration.

3 **Never run two platforms longer than you planned to.** The cost of leaving is not the hardware. It is the months in which both estates are live, both are on call, and every change has to be made twice. Pick the overlap window before you start, write it down, and treat overrunning it as the incident it is.

4 **Keep the bill until the invoice proves you can stop.** A resource is not decommissioned when nothing points at it. It is decommissioned when a full month has passed, the line has gone from the bill, and nobody has filed a ticket. Turning things off too early is how a migration becomes an outage a fortnight later.

5 **Buy the second of everything, and one spare.** Two switches, two power feeds, two people who can get into the building, and a sixth machine that is already burned in. Every single point of failure you accept at the start becomes an incident you attend in person, at night, in a year.

6 **Count the salary.** A saving that costs engineer-hours is a real saving only if you write the hours into the comparison - on both sides, because the cloud takes them too. Most of these still win by a wide margin once you do. The ones that do not, you want to know about in week one rather than month ten.

7 **Keep paying for the three things you will not beat.** A content delivery network, denial-of-service scrubbing at the edge, and outbound email deliverability. Each is a business somebody else already runs better than you will, and each is cheap next to what it replaces. Ownership is a means, not a principle.

BEFORE YOU TOUCH ANYTHING

Every Move in this book changes something that is running. Most are reversible for a stated window, a few only until a particular step, and none of them is irreversible, which is a property of this edition rather than of the work: every Move here was chosen so that it could be undone. The difference between a migration and an incident is almost never the technique - it is whether somebody worked out the way back before they started, and whether anybody had ever tested it.

READ THE ROLLBACK FIRST

Not the runbook. The Rollback section of every Move names the point of no return: the step after which the old system can no longer serve traffic or no longer holds current data. Read it, decide whether you are willing to reach that step today, and only then start at step one.

KEEP THE OLD THING RUNNING, AND KEEP PAYING FOR IT

The instinct after a successful cutover is to delete the source and book the saving. Do not. The Reversible field on each Move is how long the old system must stay warm, powered and receiving its own backups. That overlap is the entire cost of being wrong, and it is cheap.

WRITE DOWN THE STATE YOU CANNOT RECREATE

Before Stage 4, list every store whose contents exist nowhere else: the primary database, the uploads bucket, the secrets, the certificate private keys, the recovery codes for the accounts that own the domain. Everything else is rebuildable from a repository. That list is short, and it is the only part of the estate where a mistake is permanent.

TWO PEOPLE, ONE KEYBOARD

Every High-risk Move in this book assumes a second person who is not typing, is reading the runbook aloud, and has the authority to stop. It is the cheapest safety control in infrastructure and the first one teams drop when they are behind schedule. With two engineers on the whole programme, it is also the one you are most tempted to skip.

A BACKUP NOBODY RESTORED IS NOT A BACKUP

It is a file. Before Stage 4, restore the backup you are relying on into a scratch environment and count the rows against production. The number of companies that discover their backups were empty during the incident that needed them is not small, and every one of them had a green dashboard.

CUT OVER WHEN YOU CAN UNDO IT, NOT WHEN YOU ARE CONFIDENT

Confidence is not evidence. The right time to move traffic is when the rollback has been rehearsed end to end, on the same day of the week, with the same people awake. A Tuesday morning with the team at their desks beats a Saturday night with one person and a laptop, every time.

VERIFY BY COUNTING, NOT BY LOOKING

A migration is finished when the destination answers the same questions as the source with the same answers. Row counts, checksums, a replayed hour of real traffic compared response by response. A page that loads is not evidence, and neither is an application that starts.

STOP WHEN YOU ARE SURPRISED

Not when something breaks - when something is merely unexpected. A row count that is off by eleven, a certificate with the wrong issuer, a replica that caught up faster than it should have. Surprise means the model in your head and the system in front of you have diverged, and continuing is guessing.

These are operational practices, not a warranty. The runbooks in this book were written against the services and versions named in each Move and will drift as those change. Rehearse every Move against a copy of your own estate before you run it against the real one, and treat any figure here as the shape of an answer rather than a quotation for your account.

THE ARITHMETIC, WITH THE SALARY IN IT

WHAT IT ACTUALLY COSTS

Cloud ops transitions into on-prem ops, with 160 hours a month in every option. Remote hands or the rental provider covers physical work. Every column counts the salary. Substitute your own rates and hours; migration effort is separate.

ON AWS, COMPUTE ALONE

1,024 vCPU of current-generation general purpose instances, on demand, is **$37,675** a month before a single gigabyte of storage, a load balancer or a byte of egress. Egress is the line that ends most arguments: 100 TB a month is $7,980, every month, for the privilege of your own traffic leaving.

OWNED, IN COLOCATION

$9,935 of infrastructure covers metal amortised over five years, the spares, power, a full rack, transit, the cross-connect and remote hands. Add $0 of additional salaried time and $9,180 of retained services: the edge, outbound mail, scrubbing and the residue of later Moves. Those three are **$19,115** a month: what the $100,000 line becomes, base salary unchanged on both sides. Remote hands performs the agreed physical interventions; your existing team runs the platform and applications within the same monthly hours.

RENTED BY THE MONTH

$11,700 buys the reference fleet from a dedicated-host provider, with the machines, power, space, network and physical maintenance in the rental. You keep the capital available and avoid a separate facility lease. With retained services and no increase in operations hours, the cloud bill line becomes **$20,880** a month, base salary unchanged. Available stock can shorten the move. Confirm delivery, bandwidth, support and notice terms in the quote. Rent when preserving cash matters more than owning the fleet.

WHAT THE DIFFERENCE BUYS

$80,885 a month, or $970,620 a year, for a capital outlay of $248,000 — 18 machines and a pair of switches — which pays for itself in about 3 months. That is 71 per cent of a fully loaded $114,644, with the salary counted on both sides; on the infrastructure line alone, which is what every other comparison quotes, it is 81 per cent. The dollar saving is the same; counting the existing salary increases the baseline.

ONE ESTATE, THREE WAYS, PER MONTH

Every AWS figure is a public list price for us-east-1 observed while writing, before any Savings Plan, private pricing agreement or credit. Every hardware figure is a mid-market street price for the specification opposite. Both will drift. The comparison is a method you can repeat against your own invoice, not a quotation - and if your effective rate is half of list, halve the left-hand column and read the conclusion again.

THE SAME DECISION, OVER THE LIFE OF THE MACHINES

FIVE YEARS, THREE WAYS

Both metal options cut the cost of this reference estate over the life of the machines. Renting keeps the server purchase off the opening cash flow; colocation trades that purchase for lower monthly infrastructure costs. The table separates capital from running costs and counts the same operations hours in all three options, so you can choose the cash flow that suits the business.

THE SAME COMPUTERS, THREE WAYS, OVER 5 YEARS

	CAPITAL	RUNNING, A MONTH	TOTAL OVER 5 YEARS	AGAINST THE CLOUD
Cloud	–	$114,644	$6,878,613	–
Rented metal	–	$35,524	$2,131,413	$4,747,200 (69%)
Colocation	$248,000	$29,692	$1,992,293	$4,886,320 (71%)

In this model, colocation costs $139,120 less than rented metal over five years. That includes lower infrastructure costs and $37,200 of estimated fleet value at the end, counted at fifteen per cent of the purchase price. Renting preserves that opening capital and puts physical maintenance with the provider. Replace both quotes and the residual estimate with your own; location, bandwidth and support can change which route wins.

WHAT REPLACES WHAT

Every Move names the real service on all three clouds and the one thing that differs on each, so this table is the map rather than the content. Find the row you are paying for and then read the Move. The right-hand column is the point of the whole book, and where it says keep paying, that is a conclusion rather than a gap.

AWS	GOOGLE CLOUD	AZURE	WHAT YOU RUN INSTEAD
EC2	Compute Engine	Virtual Machines	Bare metal you own, or dedicated machines by the month
EC2 Auto Scaling	Managed instance groups	Virtual Machine Scale Sets	Fixed capacity, a priority class, and a spare on the shelf
EKS	GKE	AKS	Talos Linux, and Kubernetes you run yourself
ECS on Fargate	Cloud Run	Container Apps	Deployments and Jobs on your own nodes
AMI	Machine images	Managed images	A Talos machine configuration, applied over the network
VPC	VPC networks	Virtual Network	Two switches, four ranges, and an address plan on one page
EBS	Persistent Disk	Managed Disks	Ceph RBD under Rook, with local NVMe underneath it
EFS	Filestore	Azure Files	CephFS under Rook, or a filesystem the application stops needing
S3	Cloud Storage	Blob Storage	Ceph's S3 gateway - deep archive stays where it is
RDS for PostgreSQL	Cloud SQL for PostgreSQL	Database for PostgreSQL	PostgreSQL under CloudNativePG, on local NVMe
ElastiCache	Memorystore	Azure Cache for Redis	Valkey for the half that is really a cache
SQS and SNS	Pub/Sub	Service Bus	NATS JetStream, or a table in the database you already have
Glacier Deep Archive	Archive storage class	Archive tier	**Keep paying - the egress to extract it costs more than the years**
AWS Backup	Backup and DR Service	Azure Backup	Velero and the database operator's own archive, held somewhere else
Application Load Balancer	Cloud Load Balancing	Application Gateway	Envoy Gateway behind a virtual IP that fails over

WHAT REPLACES WHAT II

The rows that say keep paying are set in red because they are the most important lines in the table. A content delivery network, scrubbing capacity at the edge and outbound mail deliverability are not technical problems you have not solved yet. They are businesses, and you are not in them.

AWS	GOOGLE CLOUD	AZURE	WHAT YOU RUN INSTEAD
ACM	Certificate Manager	Key Vault certificates	cert-manager, issuing and renewing without anybody watching
Route 53	Cloud DNS	Azure DNS	Two independent authoritative providers, never one
NAT Gateway	Cloud NAT	NAT Gateway	Node addresses, and one translation hop for the few that need it
CloudFront	Cloud CDN	Azure Front Door	**Keep paying - change vendor, not model**
AWS Shield and WAF	Cloud Armor	Azure DDoS Protection and WAF	**Keep paying for scrubbing - the capacity is upstream of you**
SES	No first-party equivalent	Communication Services	**Keep paying for outbound - inbound can come home**
ECR	Artifact Registry	Container Registry	A pull-through cache, then your own registry of record
Secrets Manager	Secret Manager	Key Vault	Encrypted in the repository, with the key held outside the cluster
CodeBuild	Cloud Build	Azure Pipelines	Runners on the machines you already own and are not using
CloudWatch metrics	Cloud Monitoring	Azure Monitor	Prometheus, with a cardinality budget agreed in advance
CloudWatch Logs	Cloud Logging	Log Analytics	Loki, sized against retention somebody actually chose
CloudWatch alarms	Cloud Monitoring alerting	Azure Monitor alerts	OneUptime - alerting, on-call rotas, incidents and a status page
Cost Explorer	Cloud Billing reports	Cost Management	A model built from invoices, counting power, spares and the rota
Organizations	Organisation and projects	Subscriptions and Entra	One account left open only until the last invoice reads zero

WORK OUT WHETHER TO DO IT AT ALL

DONE WHEN You know what you spend, what you would spend instead, and whether it is worth it.

Whether to do this at all. Three Moves of arithmetic against your own invoice, and a fourth that names what stays rented. The stage most likely to end the programme, and that is a success rather than a failure.

STAGE 2 OF 5 · STAGE 2 · 4 MOVES · 4 AT ZERO DOWNTIME

ORDER THE HARDWARE AND SIGN THE SPACE

DONE WHEN The machines are on order and the cage is signed.

What to order and where to put it. Nothing here is software, and getting it wrong is the only mistake in this book you cannot fix with a deploy. Both clocks start in this stage, and they run in parallel.

TURN THE BOXES INTO A CLUSTER

DONE WHEN A cluster that could take production traffic, and never has.

Everything between a racked machine and a cluster that could take production traffic. Nothing here moves a workload; all of it decides how well the rest of the book goes.

MOVE THE APP, THEN THE DATA

DONE WHEN Everything runs on your machines, and the cloud copy is still warm.

The application first, then the state. The stage that can lose a company its data, and therefore the one with the longest overlap windows, the most rehearsal and the fewest one-way steps.

CUT THE TRAFFIC OVER, AND KEEP IT ALIVE

DONE WHEN Users reach your machines, you can carry it at 03:00, and the cloud bill is zero.

The traffic, the pager and the last account. Every article about leaving the cloud stops before this stage, and every migration that regrets itself failed inside it.

DECIDE

The invoice, the inventory and the comparison with the salary line in it. Three Moves, about a week, and permission to stop.

BUY

The specification, the machines, the space and everything with a lead time. The stage where a mistake costs a lorry rather than a deploy.

BUILD

Racking, the network, the cluster and the disks. Nothing here moves a workload, and everything here decides how the rest goes.

MOVE

Images and secrets, the first service, the buckets and the database. The longest overlaps and the most rehearsal in the book.

RUN

The edge, the cutover, the backups you have restored, the pager, and the account you finally close.

Every Move states its cutover in minutes of user-visible downtime, its risk as blast radius rather than difficulty, and how long the thing it replaced must stay warm before you turn it off. 18 of the 20 Moves in this book cost no downtime at all; the whole book, executed end to end, costs 25 minutes.

WHICH MOVE DO I NEED

Nobody opens a book like this at Move 01. They open it because something on the bill, or on the pager, has become intolerable. This is the index for the way the question actually arrives.

I

DECIDE

Whether to do this at all. Three Moves of arithmetic against your own invoice, and a fourth that names what stays rented. The stage most likely to end the programme, and that is a success rather than a failure.

4	4	0	0
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

01

THE BILL, AND THE THREE LINES THAT ARE MOST OF IT

The invoice records what you were charged, not what you use. Read it to the line, find the three that are most of it …

04

THE THREE THINGS YOU KEEP RENTING

Three capabilities do not come home: the edge, outbound mail and volumetric scrubbing. Naming them now keeps the plan …

02

WHAT YOU ACTUALLY RUN

The estate feels infinite because nobody has written it down. On one page it is forty rows, and every row has a name …

THE BILL, AND THE THREE LINES THAT ARE MOST OF IT · WHAT YOU ACTUALLY RUN · THE NUMBER THAT DECIDES IT · THE THREE THINGS YOU KEEP RENTING

3 LOW RISK | 1 MEDIUM RISK

DECIDE

01 THE BILL, AND THE THREE LINES THAT ARE MOST OF IT

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
DECIDE	—	LOW	IMMEDIATELY

> The invoice records what you were charged, not what you use. Read it to the line, find the three that are most of it, then start the clock on working set.

AWS	**Cost Explorer**	hourly and resource-level granularity is a paid opt-in, and that detail is then kept for 14 days.
GOOGLE	**Cloud Billing reports**	detailed usage cost reaches BigQuery only from the day the export is enabled, and is never backfilled.
AZURE	**Cost Management**	amortised reservation cost appears on an Enterprise Agreement account, not on pay-as-you-go, and exports land in Blob Storage on a schedule.

BEFORE YOU START

ACCESS

- Billing owner rights in every cloud account, and permission to install an agent on production instances

SOFTWARE

- The provider tools `aws`, `gcloud` and `az`, none older than six months
- A spreadsheet and a time-series store with three weeks of retention

PEOPLE

- Whoever signs the invoice, for half an hour at the end

WHY THIS WORKS

Three numbers decide everything that follows, and none is on the provider's default dashboard. A bill records what was charged — allocation, commitment, a support percentage — not what the machines were doing. Grouped into compute, state and traffic, two hundred line items collapse into three piles, and three lines usually carry eighty per cent of the spend. The window runs alongside, because the figure a purchase order needs is working set and the hypervisor cannot see it. Two weeks, not thirty days: this is a five-machine decision, and more precision would not change the order.

SWAP

If finance already breaks the bill down by service, start there and spend the saved day on step four. The categories will be wrong, the totals right.

DO IT FASTER

Pull three invoices rather than one. One month hides a reservation renewal and a support tier that stepped up.

WATCH OUT

Agent metrics are billed differently on each cloud — per unique metric a month on one, per sample ingested on the others — and an agent collecting every counter it finds has produced four-figure surprises on all three. Name the metrics you want.

LEFTOVERS

The agents, export jobs and extra ingestion bill on until somebody removes them. Diary it for the day the hardware order is signed.

—	—	—	0 min	3 days	2 weeks
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Nothing, yet. This Move moves one line upward: a fortnight of custom metrics, the cheapest few hundred dollars in the programme.

01 THE BILL, AND THE THREE LINES THAT ARE MOST OF IT

6 STEPS · 0 MIN · LOW RISK

DECIDE

THE RUNBOOK

01 Export one month at line-item granularity from every account, not the summary view. That means the detailed export rather than the console API on all three: the Cost and Usage Report delivered to a bucket, a BigQuery billing export, and `az costmanagement export` per subscription. `aws ce get-cost-and-usage` and the reports page return period totals grouped two ways at most, which is the summary view under another name.

02 Put every line in the spreadsheet, one row each, tagged compute, state or traffic: instances and containers; managed databases, caches and object storage; egress, balancers and the CDN.

03 Sort by amount and rule a line under the third row. Those three are the programme; anything beneath worth under two per cent of the invoice is not discussed again.

04 Reconcile the export against what you believe you run. Enumerate the accounts first — `aws organizations list-accounts`, `gcloud billing projects list` against each billing account, `az account list` — because the forgotten estate is usually in an account nobody named. Lines nobody owns are idle balancers, volumes of dead instances, a staging project that shipped. Expect five to fifteen per cent.

05 Install the memory agent where it is missing and point one time-series store at all three metric APIs. No hypervisor sees inside its guests, so working set arrives only from within the machine.

06 Open the window for a fortnight, and record what resists measurement: burstable credits, page cache against anonymous memory, build minutes that were never an instance.

ROLLBACK

Nothing in production changes, so backing out is removing an agent and stopping an export job. There is no point of no return here; the first is the hardware order, signed against these numbers. A sampling window cannot be rerun retrospectively, so stopping at day nine means starting over. If the time-series store is lost, restore it.

DECIDE · 0 MIN · LOW RISK

Nothing, yet. This Move moves one line upward: a fortnight of custom metrics, the cheapest few hundred dollars in the programme.

02 WHAT YOU ACTUALLY RUN

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
DECIDE	—	LOW	IMMEDIATELY

DECIDE

> The estate feels infinite because nobody has written it down. On one page it is forty rows, and every row has a name against it.

AWS	**Resource Explorer**	indexes per Region behind one aggregator and searches one account until the organisation-wide view is on, so the second account stays invisible.
GOOGLE	**Cloud Asset Inventory**	the caller needs `cloudasset.assets.searchAllResources` at the organisation scope; a project-level grant says nothing about the siblings on the same invoice.
AZURE	**Resource Graph**	reads only subscriptions the caller can already see, and pages at 1,000 rows, so an unpaged query returns a truncated estate that looks complete.

BEFORE YOU START

ACCESS

- Read-only access to every cloud account, project and subscription, including the forgotten ones
- The billing export from Move 01, at resource granularity rather than by service

SOFTWARE

- The provider's asset CLI — `aws`, `gcloud` or `az` — and one shared document with a row per resource

PEOPLE

- The engineer who would notice each service stopping, named on every row

WHY THIS WORKS

Leaving the cloud feels impossible because nobody can say what would have to move. The estate lives in three or four heads, so the total is always imagined larger than it is. A company spending $100,000 a month usually runs forty or fifty services, a handful of Postgres clusters, two or three caches, a dozen buckets and a scheduler nobody has read in a year. That still fits on a page, and every later Move then has something finite to count against. Nothing changes here: this Move reads, and writes down what it read. The risk is being wrong about what runs, and the bill settles that.

SWAP

Where accounts are few and tags honest, the provider's tag report is a quicker first pass. Untagged resources still have to be found by hand.

DO IT FASTER

Reconcile before assigning owners. The bill sorts rows by cost, which is the order worth arguing about.

WATCH OUT

These inventories list what the provider created. What a third party stood up against your account — an agent, a contractor's sandbox — appears nowhere else.

LEFTOVERS

A snapshot outlives the volume it came from and bills long after the rest of its account has gone. So does a retained log group.

WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT
—	—	—	0 min	4 days	—

TURN OFF The fourth pile, once its owners confirm: orphaned volumes, snapshots of vanished disks, idle load balancers and the proof-of-concept accounts. The cheapest saving in the book.

NEEDS 01 The bill, and the three lines that are most of it

02 WHAT YOU ACTUALLY RUN

5 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Pull the machine-readable list, one account at a time: `aws resource-explorer-2 search`, `gcloud asset search-all-resources` or `az graph query`. Paste it into one document, a row per resource.

02 Add what those APIs never return: scheduled jobs, queues, third-party services with a webhook into production, and the DNS records pointing at any of it.

03 Reconcile against the billing export from Move 01 both ways. A bill line with no row is unowned; a row with no bill line is free, or in an account nobody has opened.

04 Hunt what hides: snapshots whose disk is gone, volumes attached to nothing, load balancers with no target, staging at full size overnight, the accounts opened for a proof of concept in 2022.

05 Sort every row into one of four piles — moves as-is, moves with work, stays rented, gets deleted. The fourth is the biggest; confirm each row with its owner and check thirty days of metrics first.

ROLLBACK

Nothing here touches a running system, so backing out is discarding a document nobody acted on. The point of no return is the fourth pile: once a resource on it is deleted, getting it back means a restore from whatever snapshot survives, and for a log group there may be none. Keep the inventory in version control.

DECIDE · 0 MIN · LOW RISK

The fourth pile, once its owners confirm: orphaned volumes, snapshots of vanished disks, idle load balancers and the proof-of-concept accounts. The cheapest saving in the book.

03 THE NUMBER THAT DECIDES IT

CUTOVER
0 MIN

0 — 60

LAYER	LEAVING	RISK	REVERSIBLE
DECIDE	—	MEDIUM	IMMEDIATELY

DECIDE

> Three columns, the same operations team and five years of costs. Put colocation and rented metal beside the cloud bill and choose the margin worth moving for.

AWS	**AWS Pricing Calculator**	two tools now carry that name, and only the one inside the billing console can see the Savings Plans your account already holds.
GOOGLE	**Google Cloud Pricing Calculator**	the only discount it models is a committed-use one you pick from a dropdown, and its default machine series earns no sustained-use discount at all.
AZURE	**Azure Pricing Calculator**	signing in and selecting the Enterprise Agreement licensing programme rewrites every price on the page, so an anonymous quote is retail and not yours.

BEFORE YOU START

ACCESS

- Three recent invoices at the rate you actually paid, not list
- Quotes for sixteen active machines, two spares and a full rack, plus an equivalent dedicated-server rental

SOFTWARE

- One spreadsheet, three columns: cloud, rented metal and colocation

PEOPLE

- The existing cloud operations lead, to map the team's work onto the new platform

WHY THIS WORKS

Moves 01 and 02 produced a grouped bill and an inventory that accounts for it. This Move compares three ways to run that estate: the cloud, rented dedicated servers and owned hardware in colocation. Both metal options can keep the existing operations team while reducing the infrastructure bill; renting also keeps the hardware purchase off the opening budget. Put support, retained services and one-time migration work into the comparison, then apply a margin chosen before the figures. Two days of analysis gives the team a decision it can fund and defend.

SWAP

Price the three largest lines only; that lands within about ten per cent in an afternoon.

DO IT FASTER

Use one sheet with the same capacity, support and retained-service assumptions in every column; compare it with the cloud rate you actually pay.

WATCH OUT

Equal recurring hours assumes a team that already operates production, a standardised platform and contracted physical support. Check that scope, then vary hours, rental rates and hardware life in the sheet. A migration task becomes a monthly cost only if it actually recurs.

LEFTOVERS

The committed-spend agreement bills to the end of its term, and that renewal date is the deadline.

WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT
—	—	—	0 min	2 days	—

TURN OFF Nothing yet. This Move produces a number and, with it, permission to act or to stop.

NEEDS 01 The bill, and the three lines that are most of it 02 What you actually run

03 THE NUMBER THAT DECIDES IT

7 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Write the decision rule in the spreadsheet before any figure goes in: a metal option must land at least a third under the current cloud cost to justify this programme. Apply the same margin to rented metal and colocation.

02 Fill the cloud column from the three invoices at the rate you actually paid, discounts in and tax out across all columns. One monthly number, checkable against Move 01.

03 Fill the colocation column from the hardware and facility quotes: sixteen machines and two spares over sixty months, the full rack, power, transit, cross-connects and remote hands. Beside it, quote the equivalent rented fleet with its traffic allowance and hardware support included.

04 Carry the existing operations salary into all three columns. This reference model budgets the same 160 hours a month for each: cloud ops transitions to platform ops, with physical work assigned to the facility's remote hands or the rental provider under the support contract. Validate that assumption with the pilot and the people doing the work. Price setup, training and migration once in the project budget; any measured recurring difference belongs in the monthly line.

05 Add the services that stay rented. Move 04 identifies the edge, outbound mail and scrubbing; later Moves retain archived object storage, a registry, a queue and an off-site backup copy. Their cost is already in the cloud invoice and must remain in both metal columns.

06 Total the three columns and apply the rule. If either metal option clears it, compare the five-year totals and the monthly cash requirement. If neither clears, keep the sheet and revisit it when the estate or quotes change.

07 Choose how to fund the move. Colocation buys the fleet and carries its value forward; rented metal pays monthly and keeps that capital available. Put the actual purchase quote, one-time migration effort and overlap with the cloud bill in front of whoever signs. Move 07 settles the facility and rental terms.

ROLLBACK

Nothing is ordered and nothing signed, so this Move reverses by changing a cell. The point of no return is the order in Move 06 and the contract in Move 07. Keep dated versions of the sheet, so earlier assumptions can be restored when somebody asks how the decision was reached.

DECIDE · 0 MIN · MEDIUM RISK

Nothing yet. This Move produces a number and, with it, permission to act or to stop.

04 THE THREE THINGS YOU KEEP RENTING

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
DECIDE	NOTHING — THIS MOVE DECIDES WHAT STAYS RENTED	LOW	IMMEDIATELY

> Three capabilities do not come home: the edge, outbound mail and volumetric scrubbing. Naming them now keeps the plan honest.

AWS	**CloudFront and Shield**	Shield Advanced bills $3,000 a month on a twelve-month term and attaches only to AWS resources, so it protects a distribution and never a rack.
GOOGLE	**Cloud CDN and Cloud Armor**	Cloud CDN is a mode on a load balancer backend, so a colocated origin must first become an internet network endpoint group.
AZURE	**Front Door and its web application firewall**	managed rule sets need the Premium tier, and the firewall policy is a resource that outlives its profile.

BEFORE YOU START

ACCESS

- The three invoice lines for edge, mail and mitigation, and authority to sign

SOFTWARE

- The comparison from Move 03, open for a new permanent line
- The sending domain's current SPF, DKIM and DMARC records

PEOPLE

- Whoever owns the marketing sends, who shares the reputation
- An engineer willing to name the fourth item honestly

WHY THIS WORKS

A plan that pretends everything comes home fails in Stage 5, when somebody notices password resets landing in spam folders and no schedule is left. Three capabilities are bought rather than built, because building them badly is worse than renting them well: a hundred points of presence, a decade of sender reputation, and absorption capacity upstream of your transit. None is expensive next to what it prevents, and all three are worth paying for after the move on the terms you pay now. Deciding here costs two days and removes the largest late surprise.

SWAP
One vendor covering delivery and mitigation halves the contracts, and one outage then takes both.

DO IT FASTER
Ask each vendor for the contract and onboarding checklist in the first message.

WATCH OUT
Facility-included protection is often a null route with a better name. Get in writing what it does to your address.

LEFTOVERS
The cloud edge stays in front of the cloud origin until Move 18 shifts traffic, so two edge bills overlap.

$5,200/mo	$5,200/mo	0%	0 min	2 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Nothing, and that is the point. The cloud's versions switch off in Moves 18 and 20; these three stay on the invoice permanently.

NEEDS 02 What you actually run 03 The number that decides it

04 THE THREE THINGS YOU KEEP RENTING

5 STEPS · 0 MIN · LOW RISK

THE RUNBOOK

01 Price the edge. Take the delivery and egress figures off the invoice from Move 01 and quote three independent vendors against that volume. A content network in front of a rack is the same arrangement as in front of the cloud.

02 Price outbound mail on its own. Transactional sending goes to a provider whose only business is deliverability, and the sending domain's SPF, DKIM and DMARC records point at it. Marketing sends take a separate subdomain.

03 Price mitigation upstream of your transit, the only place it works. Ask the facility and each transit vendor what is included, at what capacity and how it triggers: a hundred-gigabit attack against a ten-gigabit port is decided before it reaches your equipment.

04 Sign all three contracts before the hardware ships, confirming each notice period and minimum term. Mail warm-up takes weeks and cannot begin during an incident.

05 Write the total, about $5,200 a month, into the comparison from Move 03 as a permanent line. Then name the fourth item: whatever the business depends on that nobody here has run, such as a payments integration.

ROLLBACK

Nothing here touches a running system, so backing out is cancelling inside a notice period. The point of no return arrives later, when the sending domain's records point at the new provider: reputation then accrues on their addresses, and the old standing cannot be restored by changing the records back.

DECIDE · 0 MIN · LOW RISK

Nothing, and that is the point. The cloud's versions switch off in Moves 18 and 20; these three stay on the invoice permanently.

STAGE 2 · SIZING, THE MACHINES, THE CAGE, AND EVERYTHING WITH A LEAD TIME

II

BUY

What to order and where to put it. Nothing here is software, and getting it wrong is the only mistake in this book you cannot fix with a deploy. Both clocks start in this stage, and they run in parallel.

4	4	2	0
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

05

FROM RENTED VCPUS TO CORES YOU OWN

Two vCPUs is one physical core, and the second thread is worth about a quarter of the first — the gap between sixteen …

06

SIXTEEN MACHINES, AND THE TWO ON THE SHELF

Sixteen nodes, so losing one costs six per cent and not a third. Two are bought with them and never plugged in …

FROM RENTED VCPUS TO CORES YOU OWN · SIXTEEN MACHINES, AND THE TWO ON THE SHELF · A CAGE, NOT A DATA CENTRE · THE ORDER, AND THE WEEKS YOU CANNOT COMPRESS

2 MEDIUM RISK

2 HIGH RISK

05 FROM RENTED VCPUS TO CORES YOU OWN

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
BUY	THE VCPU AS A UNIT OF PURCHASE	MEDIUM	UNTIL THE ORDER IS SIGNED

> Two vCPUs is one physical core, and the second thread is worth about a quarter of the first — the gap between sixteen machines and thirty.

BUY

AWS	**EC2 instance types**	a vCPU is one SMT thread on the x86 families but a whole physical core on Graviton, labelled the same.
GOOGLE	**Compute Engine machine types**	custom shapes set memory apart from cores, within a per-core band that drags cores onto the bill.
AZURE	**Virtual Machine sizes**	the B-series meters CPU against a credit balance, so a saturated workload reads low once its credits run out.

BEFORE YOU START

ACCESS

- The two-week measurement window from Move 01, working set sampled apart from CPU
- The instance inventory from Move 02, exported with family and size

SOFTWARE

- A sizing sheet other people can check, with the derate and headroom visible
- The vendor documentation for each instance family you run

PEOPLE

- Whoever signs the order in Move 06, present for the derate rather than shown it

WHY THIS WORKS

Move 01's samples become a part number here. The first conversion is the one nobody writes down: on x86 a vCPU is one hyperthread, so two share a physical core, and the second thread is worth fifteen to thirty per cent of the first, not another hundred. A 96-vCPU estate is a 48-core estate, which fits in two of these machines. Then three rules in order: memory, the ceiling a node reaches; clock, because a query plan runs mostly on one core; flash, because local NVMe beats rented block storage by orders of magnitude.

SWAP

With the window unfinished, size against last quarter's billed peak and mark the sheet provisional. It lands a third high.

DO IT FASTER

Most estates run five workload shapes wearing forty names. Cluster them and the conversion takes an afternoon.

WATCH OUT

Rightsizing advice optimises for a smaller rented instance, a different question from how much silicon to own.

LEFTOVERS

Local NVMe delivers the provisioned IOPS line as a property of the drive. Check it against your measured write rate.

WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT
—	—	—	0 min	3 days	—

TURN OFF Nothing yet, though the re-measurement in step two usually finds a service two sizes above its demand — shrink it and the bill falls this month.

NEEDS 01 The bill, and the three lines that are most of it 03 The number that decides it

THE RUNBOOK

01 Group the instance inventory by family and mark each thread-per-vCPU or core-per-vCPU against the vendor documentation. On Graviton and the other Arm shapes a vCPU is already a whole core, and halving buys double.

02 Pull the burstable families out of the measurement window. `B-series` and `t3` in standard mode meter against a credit balance, and once the balance is empty a saturated workload pins at its baseline — twenty per cent on a `t3.medium`. `t3` defaults to unlimited mode, where the number is honest and the burst above baseline arrives on the invoice instead. Re-measure either way before the figure reaches the sizing sheet.

03 Halve the thread groups and credit the second thread with the fraction it is worth. Record which peak you sized against: you paid for peak every hour of the month, and owned iron carries peak by definition.

04 Size memory from the working set plus kubelet reserve, page cache and the eviction threshold, then round up to a population that fills every channel. A node cannot be widened while it runs.

05 Take clock over count on the latency path — most of a Postgres query plan executes on one core — then turn capacity and write rate into drives and hand the specification to whoever signs the order in Move 06.

ROLLBACK

A specification is undone by writing a better one: change the derate and reissue it. Nothing has been bought. The point of no return is the signature in Move 06, after which a short memory line is a second purchase with a lead time. Keep the sheet in version control so a revision can be restored.

BUY · 0 MIN · MEDIUM RISK

Nothing yet, though the re-measurement in step two usually finds a service two sizes above its demand — shrink it and the bill falls this month.

06 SIXTEEN MACHINES, AND THE TWO ON THE SHELF

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
BUY	ELASTIC NODE CAPACITY AND CLUSTER AUTOSCALING	HIGH	UNTIL THE ORDER IS SIGNED

> Sixteen nodes, so losing one costs six per cent and not a third. Two are bought with them and never plugged in, because iron does not autoscale

BUY

AWS	**EC2 Auto Scaling groups**	the fleet ceiling is a per-region vCPU quota, raised by support ticket.
GOOGLE	**Managed instance groups**	the instance template is immutable, so a shape change means a rolling replace.
AZURE	**Virtual Machine Scale Sets**	the platform fault domain count is fixed at creation and cannot be edited.

BEFORE YOU START

ACCESS

- The per-node sizing from Move 05 and the service inventory from Move 02
- Authority to commit the purchase or rental budget chosen in Move 03

SOFTWARE

- A quote comparison sheet with three vendors or rental providers priced line by line

PEOPLE

- A vendor or provider contact who will quote complete machines and support
- Whoever signs the purchase or rental commitment, with the chosen route agreed before ordering

WHY THIS WORKS

Fleet size is the quietest thing the bill decides. Three nodes are a quorum and nothing more: lose one and a third of the capacity goes with it. Sixteen and a dead machine costs six per cent, which is the difference between an incident and a note in the channel — and it is why a repatriation at this size is a far easier argument than the same one at a tenth of the bill. Buy the headroom the arithmetic already bought you: 512 cores against the four hundred or so the estate actually needs. For owned colocation, two spare machines are built, burned in and configured with the active fleet, then unplugged and shelved. Remote hands can put one into service while a failed machine goes through repair or return. The rented route reserves the same two machines as spare capacity in its eighteen-server quote; agree the provider's replacement response and how that capacity is brought into service before ordering.

SWAP

Lease the eighteen machines over three years. The total is higher, but the monthly line resembles the cloud bill it replaces.

DO IT FASTER

Ask for a configuration the vendor stocks. Wanting 288 GB because the model said 240 turns a two-week delivery into an eight-week build.

WATCH OUT

Rails, power leads, 25 GbE optics and the support contract are quoted separately. A bill of materials listing only servers is short by thousands.

LEFTOVERS

A spare helps only if it stays current. Firmware and the Talos version drift within a quarter, so the shelf machine boots and updates with the rest.

—	—	—	0 min	3 days	4 weeks
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Nothing for months. The EC2 Auto Scaling groups carry production until Move 18 shifts the traffic; this Move commits the selected purchase or rental budget.

NEEDS 02 What you actually run 05 From rented vCPUs to cores you own

06 SIXTEEN MACHINES, AND THE TWO ON THE SHELF

5 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Put the per-node sizing from Move 05 beside the service inventory from Move 02 and divide by what a node holds, leaving capacity for an active node to fail. The reference estate uses sixteen active nodes; the two spare machines are additional replacement capacity.

02 Write the specification on one page: dual-socket, 32 physical cores, 256 GB of ECC memory with every channel populated, four 3.84 TB mixed-use NVMe drives plus a mirrored boot pair, redundant power supplies, two 25 GbE ports, and a controller speaking Redfish.

03 Send that page to three vendors for colocation, or three dedicated-host providers for rented metal. Record complete capacity, support and delivery terms in the quote comparison sheet. For a purchase, mark which components may be refurbished and quote new drives and power supplies; for rental, confirm that the provider supports your operating-system image and remote management.

04 Include two identical spare nodes alongside the sixteen active ones. On the owned route, burn them in with the fleet and leave them on the shelf for remote hands. On the rented route, reserve two servers for replacement capacity and document how the team activates them while the provider repairs a fault. Keep all eighteen machines in the cost comparison.

05 Confirm the route chosen in Move 03 against the hardware and facility quotes or the rental quote before making a binding commitment. For owned colocation, have the capital approver sign the hardware purchase order and record the ship date. For rented metal, order the quoted fleet under the agreed support and notice terms; the provider supplies the facility, so no hardware purchase or separate cage order is needed.

ROLLBACK

Before an order is accepted, both routes can be reversed by leaving the quote unsigned. The point of no return is the binding commitment: configured hardware may be non-returnable, while a rented fleet can carry minimum-term and notice charges. Check those terms before signing. An owned fleet can be resold at its then-current value; a rental ends under the provider's cancellation terms.

BUY · 0 MIN · HIGH RISK

Nothing for months. The EC2 Auto Scaling groups carry production until Move 18 shifts the traffic; this Move commits the selected purchase or rental budget.

07 A CAGE, NOT A DATA CENTRE

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
BUY	THE REGION-AND-ZONE ABSTRACTION	HIGH	UNTIL THE CONTRACT IS SIGNED

> The facility supplies power, cooling, connectivity and remote hands. Your team operates the platform; rented metal includes the room and the machines.

AWS	**Regions and Availability Zones**	zone letters are shuffled per account, so one account's us-east-1a is another's us-east-1c, and cross-zone traffic bills both ways.
GOOGLE	**regions and zones**	a subnet covers the whole region rather than a zone, and quota is granted per region, so capacity in one zone is not capacity in the next.
AZURE	**regions, availability zones and zone-redundant tiers**	zone support varies by region and by machine series, and a resource created without a zone cannot be moved into one later.

BUY

BEFORE YOU START

ACCESS

- The committed power figure and machine count from Move 06
- Quotations from three facilities against one written specification

SOFTWARE

- A comparison sheet with monthly charge, cross-connect price and notice period side by side

PEOPLE

- A remote hands team with an agreed response time, plus two internal contacts authorised to direct work
- Whoever signs the term, before the price is agreed

WHY THIS WORKS

Colocation puts the building, power, cooling and physical support under one facility contract. Remote hands can inspect equipment, replace labelled parts and move cables to an agreed runbook while your existing operations team manages the platform remotely. This Move adds a facility charge — about $4,500 a month for space, transit, cross-connects and hands before usage-based power — in exchange for that support. Rented metal bundles the room and machines into the monthly server price. Price both routes before signing; either lets the team operate infrastructure without running a data centre.

SWAP

Rented dedicated servers can shorten procurement and avoid a separate cage contract. Confirm delivery dates, private networking, address portability and replacement response before ordering. The reference budgets the same operations hours for both routes: remote hands covers contracted physical work in colocation, and the rental provider covers it on dedicated servers. Choose renting when keeping capital available matters more than owning the fleet.

DO IT FASTER

Ask for the standard agreement and the cross-connect list before price is discussed; a day of reading decides which clauses are worth arguing.

WATCH OUT

A second building doubles the hardware, the spares and the change surface, and is the commonest reason a repatriation stalls. One site with backups held elsewhere beats two nobody has exercised.

LEFTOVERS

The cage invoices from handover while the cloud bill stays where it was; both run in parallel for about four months.

WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT
—	$4,500/mo	—	0 min	4 days	3 weeks

TURN OFF Nothing. This Move adds an outgoing instead of removing one; the cloud bill does not move until the first service does.

NEEDS 03 The number that decides it

THE RUNBOOK

01 Write one specification — full rack, committed kilowatts, two feeds, cross-connects, remote hands, term — and send it to three facilities. Set the location around network latency, delivery access and the contracted physical response time.

02 Size the power commitment against what actually draws: sixteen active nodes at about 450 W draw 7.2 kW before switches. The reference starts with a 10 kW commitment; verify the switch load and the facility's continuous-load limit, with either feed able to carry the full rack if the other fails. Leave the two spares unplugged on the shelf.

03 Contract the remote hands scope: inspections, labelled disk and machine replacements, cabling and power actions under an approved runbook. Put ticket-to-touch, included hours, out-of-hours rates and a named escalation in writing. Give the team the rack map and spare-part locations; name two internal contacts who can authorise work remotely.

04 Ask which carriers have live fibre in the meet-me room and which of them will sell at your volume, then get the cross-connect price list in writing, installation and disconnection included. Three carriers and a cheap cross-connect beat better marketing.

05 Negotiate the exit with the entrance: a term no longer than the hardware life, a capped escalator, a notice period you could serve. Three years with twelve months' notice repeats the mistake you are here to correct.

06 Before anyone signs, price the alternative in the next column: the same eighteen machines rented by the month, with power, networking and hardware replacement terms specified. Compare current quotes at matching capacity and support. Rented metal removes the fleet purchase and the separate facility agreement; use Move 03's full-cost comparison to choose the route.

ROLLBACK

Refusing a facility that will not put ticket-to-touch in writing costs nothing: before signature there is no position to unwind. The point of no return is the commencement date, after which departure runs on the notice period rather than on a decision. File the quotations that lost; at renewal they restore a price to argue from.

BUY · 0 MIN · HIGH RISK

Nothing. This Move adds an outgoing instead of removing one; the cloud bill does not move until the first service does.

08 THE ORDER, AND THE WEEKS YOU CANNOT COMPRESS

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
BUY	**PROVIDER-ASSIGNED ADDRESSES AND MANAGED TRANSIT**	MEDIUM	UNTIL THE ORDER IS SIGNED

> Everything with a lead time goes on one order this afternoon, so the queues run beside each other instead of one after another.

AWS	**Elastic IP and Direct Connect**	the cross-connect authorisation expires ninety days after issue, and an Elastic IP bills on after its instance is gone.
GOOGLE	**external IP and Cloud Interconnect**	the partner variant hides the physical layer, so there is no cross-connect to order and capacity comes from a fixed list.
AZURE	**Public IP and ExpressRoute**	the circuit bills from creation, before the carrier has provisioned anything against its service key.

BEFORE YOU START

ACCESS

- The signed facility agreement from Move 07 and its cross-connect price list
- Purchase authority to sign transit and circuits

SOFTWARE

- The switch part numbers from Move 06, for the optic compatibility list
- A written order sheet, one promised date and one name per line

PEOPLE

- A named technical contact the carriers can reach in week three

WHY THIS WORKS

Every other stage of this migration is work you control. This one is queueing, and the only thing you control is when it starts. A cross-connect and an optic ordered in the same week arrive while the machines are being built; ordered in sequence they add a month each. So the whole list goes in on one afternoon, and you accept paying for two transit paths from the start, because one upstream is a single point of failure with a contract attached. Nothing is installed yet, so every line can be withdrawn up to signature.

SWAP

Buy every line from the facility on one invoice. It costs more per megabit and takes four vendors off the critical path.

DO IT FASTER

Ask for the authorisation templates before the agreement is countersigned; no queue starts until they are lodged.

WATCH OUT

A cloud's interconnect paperwork expires if the cross-connect is not done inside ninety days, and it is re-issued rather than extended.

LEFTOVERS

The blended transit stays on the invoice until your own circuit has carried a full month including a peak.

WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT
–	–	–	0 min	3 days	4 weeks

TURN OFF Nothing yet — this Move only adds lines. The first it retires is the blended transit, once your own upstream has held a month.

NEEDS 07 A cage, not a data centre

08 THE ORDER, AND THE WEEKS YOU CANNOT COMPRESS

5 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Order the cross-connects the day the facility agreement is countersigned: one per upstream, one for the out-of-band line, one spare. Ask whether the panels to the meet-me room are in place; that decides days against weeks.

02 Take the facility's blended transit for go-live, already in the building, and order a second upstream from a carrier on a different fibre route to run beside it. On the facility's own addresses that second circuit buys a price and a path to grow into rather than a failover, because only the facility's transit routes to you; it becomes a failover the day you announce your own prefix to both.

03 Take a /24 or a /25 and IPv6 from the facility. Your own autonomous system number can wait until a second site earns its eight weeks of registry paperwork. Rented space is a renumbering you may do twice.

04 Order the out-of-band line: a small separate circuit, or a mobile router. It is the cheapest item here and the one that matters at three in the morning.

05 Order optics and cables from the switch vendor's supported list, twenty per cent over the port count. Then chase the order sheet weekly.

ROLLBACK

Until a signature this Move is undone by an email. The point of no return is the first signed term, not the first delivery: transit and cross-connects carry minimum terms in months, and a circuit against the wrong cabinet is paid for whether or not it is lit. Keep the quotes in version control, so a position can be restored at renewal.

BUY · 0 MIN · MEDIUM RISK

Nothing yet — this Move only adds lines. The first it retires is the blended transit, once your own upstream has held a month.

STAGE 3 · RACKING, THE NETWORK, THE CLUSTER AND THE DISKS

III

BUILD

Everything between a racked machine and a cluster that could take production traffic. Nothing here moves a workload; all of it decides how well the rest of the book goes.

4	4	3	0
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

09

RACKING DAY

Eighteen boxes become eighteen machines reachable from a hotel room. Rails, power and labels take the day; the proof …

12

DISKS: WHAT GOES LOCAL, WHAT GOES ON CEPH

Ceph takes three drives a node for anything that must survive a node dying. The fourth stays raw under PostgreSQL …

10

THE NETWORK, AND THE WAY BACK IN WHEN IT BREAKS

Four address ranges written down before anything is configured, two switches either of which may fail, and a way in …

1 MEDIUM RISK | 3 HIGH RISK

09 RACKING DAY

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
BUILD	THE PROVIDER'S SERIAL CONSOLE AND BOOT DIAGNOSTICS	MEDIUM	IMMEDIATELY

> Eighteen boxes become eighteen machines reachable from a hotel room. Rails, power and labels take the day; the proof takes twenty minutes.

AWS	**EC2 Serial Console**	granted per account, Nitro types only, and no virtual media, so your own image never boots.
GOOGLE	**Compute Engine serial console**	switched on by the `serial-port-enable` metadata key, and an organisation policy can forbid it estate-wide.
AZURE	**Boot diagnostics and Serial Console**	on by default against a managed storage account you cannot open, gated behind a role and killable subscription-wide, and reaches the guest only, never wedged firmware.

BUILD

BEFORE YOU START

ACCESS

- A booked day on site for two named people
- An out-of-band line that is live and independent of the uplinks

SOFTWARE

- `ipmitool` and a Redfish client on a laptop outside the building
- A memory tester, a drive exerciser and one firmware bundle for the whole batch

PEOPLE

- Two people for the lift; a chassis at head height is not a one-person job

WHY THIS WORKS

The physical work is ordered so nothing has to be undone: rails across all six positions before a chassis is lifted, weight low, power split so one distribution unit cannot take the estate. None of that is the point. The Move exists for the baseboard management controller in each node — a small computer with its own processor, port and firmware, and the whole of the console the cloud gave you. Proved from outside the building, it turns two years of faults into an afternoon at a keyboard rather than a drive to the facility.

SWAP

Remote hands will rack and cable to a drawing at about $150 an hour; buy that for the lifting, not the controllers.

DO IT FASTER

Address and password the controllers on a bench the week before, so the day itself is cabling and testing.

WATCH OUT

Controllers answer on any interface they find; one reachable from the internet is a full-power console for a stranger.

LEFTOVERS

Cardboard, the spare's rail kit, and the temporary address scope that hands out addresses for months.

–	–	–	0 min	6 days	–
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Nothing yet. Eighteen machines draw power in one building while the whole cloud bill runs in another.

NEEDS 06 Sixteen machines, and the two on the shelf 08 The order, and the weeks you cannot compress

09 RACKING DAY

6 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Fit all six rail pairs before any chassis goes in, checked against the cabinet's depth. Rack bottom-up, heaviest low, two people on each lift.

02 A-side power from one distribution unit, B-side from the other, so a node's supplies never share a strip. Pull one feed per node and confirm it stays up.

03 Cable data in one colour and management in another, both ends labelled before anything is plugged in. Label the position, not the machine.

04 Give each controller a static address on the management VLAN, serial-over-LAN on, and a password the cluster cannot reach. Run each operation through `ipmitool` and Redfish, noting which answers.

05 Leave the building. From a laptop elsewhere, over the out-of-band line, watch a node POST, enter firmware setup, mount an image over virtual media and power-cycle it. Until that works from outside, nothing here is built.

06 Run the memory tester and the drive exerciser overnight on all eighteen nodes, level them all to one firmware bundle, and before leaving site take front and rear photographs, the port map and a file naming which node is in which rack unit.

ROLLBACK

Nothing here carries data: a mistake is undone by unracking a machine the same afternoon. The point of no return is the day this cabinet takes production traffic, after which moving a cable is a change window. Firmware is the exception — a downgrade is sometimes refused, so record the shipped version per serial and keep the vendor's image so a working level can be restored.

10 THE NETWORK, AND THE WAY BACK IN WHEN IT BREAKS

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
BUILD	CLOUD-MANAGED PRIVATE NETWORKING	HIGH	IMMEDIATELY

> Four address ranges written down before anything is configured, two switches either of which may fail, and a way in that depends on neither.

AWS	**VPC and its subnets**	a subnet is pinned to one Availability Zone and its block is fixed at creation, and frames run at 9001 bytes inside the VPC but 1500 through the internet gateway.
GOOGLE	**VPC networks and subnets**	the MTU belongs to the network rather than the instance, and every virtual machine must be restarted before it uses a changed value.
AZURE	**Virtual Network and its subnets**	no broadcast or multicast on the wire, so gratuitous ARP and VRRP never worked and a floating address had to be a load balancer.

BUILD

BEFORE YOU START

ACCESS

- Console access to both switches, and the out-of-band line from Move 09 answering
- The public address range from Move 08, and the private ranges the organisation already uses

SOFTWARE

- `ipmitool` 1.8.19 or newer, to prove the baseboard controllers answer on the management range
- A packet test that sets the do-not-fragment bit, because a default-size ping proves nothing here

PEOPLE

- Two engineers for the switch-pull drill, one at the rack and one watching the graphs

WHY THIS WORKS

The address plan is cheap to get right today and expensive to change in a year, because by then it sits in a partner's firewall and on every machine. It is written first, on paper: four ranges, each with twice the room the estate needs now. The switches come as a pair, since one switch is one maintenance window that takes everything down with it. MTU gets a step of its own, because a mismatch does not fail cleanly — it passes small requests and stalls large ones. Nothing here carries production traffic yet, which is what makes the week safe.

SWAP

Where the switches will not do multi-chassis aggregation, give each node an address per uplink and let it route to both independently: more host configuration, fewer shared failure modes.

DO IT FASTER

Generate both switch configurations from the address plan rather than typing them twice. Hand-built pairs differ in one port, and the drill is what finds it.

WATCH OUT

The way back in must not depend on the fabric it exists to rescue. A jump host on the data VLAN, or a mesh agent that needs the production default route, is neither.

LEFTOVERS

The cloud private network and its per-gigabyte charges keep running throughout, and nothing here removes them.

—	—	—	0 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Nothing yet. The cloud's private network carries every service you run until the traffic moves in Stage 5.

NEEDS 09 Racking day

10 THE NETWORK, AND THE WAY BACK IN WHEN IT BREAKS

6 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Write the address plan before touching a switch. Four ranges — management, storage, node addressing and the public block from Move 08 — none overlapping the office, the cloud accounts or anything a partner routes. It lives in source control and nowhere else.

02 Cable the two top-of-rack switches as a multi-chassis pair with a peer link, and run two uplinks from every node into different switches, bonded. One switch then fails as a component rather than as an outage.

03 Build the management VLAN with no route to the internet. The baseboard controllers and the out-of-band circuit sit on it, and the way in is a jump host, never a public address. Prove it from outside the building with an `ipmitool` power status on every node.

04 Set the MTU identically on every switch port, every bond and every node, and record the number beside the address plan. Undocumented mixed segments are how a fortnight disappears.

05 Test with full-size packets that refuse fragmentation, on every path that will carry traffic. A mismatch breaks about five per cent of it and reads as an application bug for a week.

06 Run the drill in daylight, with both engineers watching. Pull the power from one switch, confirm every bond keeps a member, then plug it back in and confirm the link rejoins. Nobody who declines this today will do it under load.

ROLLBACK

Nothing here carries production traffic, so backing out is reverting one switch configuration and restoring the previous one from source control. The point of no return is a date rather than a command: once node addresses and partner firewall entries reference the plan from step one, renumbering becomes a weekend rather than an edit. Argue the ranges into shape now, not after Move 11.

BUILD · 0 MIN · HIGH RISK

Nothing yet. The cloud's private network carries every service you run until the traffic moves in Stage 5.

11 A CLUSTER, IN AN AFTERNOON

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
BUILD	THE MANAGED KUBERNETES CONTROL PLANE	HIGH	IMMEDIATELY

> The frightening part is the short part: eighteen machines, one file in Git, and an operating system with no shell to log into.

AWS	**EKS**	tearing a cluster down leaves its IAM OIDC provider behind, still trusting an issuer URL nothing resolves.
GOOGLE	**GKE**	the control plane upgrades on its release channel regardless; a full-freeze exclusion cannot exceed 90 days, and holding a minor version only lasts until that version leaves support.
AZURE	**AKS**	the Free tier carries no API server uptime guarantee at all, so what is given up is a paid Standard tier or nothing at all.

BUILD

BEFORE YOU START

ACCESS

- The eighteen machines from Move 10, powered and reachable on the management VLAN
- One unused address on the server VLAN, for the API virtual IP

SOFTWARE

- `talosctl` and `kubectl` at the same minor version as the cluster you are building
- `helm` and the Cilium chart pinned to an exact chart version, not a range
- The `cilium` CLI, for the connectivity test

PEOPLE

- Someone to power a machine off at the wall while another watches the API

WHY THIS WORKS

A control plane is three machines that agree with each other, and the agreement is etcd. Talos Linux removes the reason hand-built clusters rot: no shell, no package manager, no login, so changing a machine means changing the YAML in Git and applying it. Drift stops being a category. Doing this while nothing runs is what makes the risk survivable — every mistake costs a rebuild, and an empty cluster rebuilds in twenty minutes. Be plain about the money: a managed control plane is about $73 a month each and you run three — the same ten cents an hour at all the providers — so this is not the Move that pays for the migration. Everybody arriving from EC2 asks why there is no hypervisor — why this sits on the metal rather than on Proxmox, with as many virtual machines as you like. Because you are not moving virtual machines: Move 02's inventory is already containers, and a hypervisor under a container platform is a second control plane to patch, licence and back up for a problem this estate does not have. It also puts back the shell that Talos removes, which is what the ongoing-hours figure assumes you did not do. The Swap says when to take the other branch.

SWAP

If replacing kube-proxy on day one is a step too far, run Cilium alongside it and take kube-proxy out a week later.

SWAP

Proxmox underneath, cluster in virtual machines, is right in three cases: workloads that are not containers and never will be; rebuilding or resizing a node without walking to the rack; or snapshotting a whole machine before an upgrade rather than only its workloads. The price is a second platform to patch. Take it deliberately, not by habit.

DO IT FASTER

Build the cluster twice from the same commit. The second build takes twenty minutes and shows whether the first was reproducible or lucky.

WATCH OUT

On flash without power-loss protection, etcd throws leader elections under write load, which reads as an intermittent network fault.

LEFTOVERS

Write the upgrade cadence down today — one minor behind, reviewed quarterly. Nobody picks it calmly once a version is out of support.

$220/mo	$0/mo	100%	0 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Nothing yet — the managed cluster still carries production and stays until Move 20. Its per-hour control-plane charge is the first bill line to reach zero.

NEEDS 10 The network, and the way back in when it breaks

11 A CLUSTER, IN AN AFTERNOON

6 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Commit the machine configuration before applying it. Generate the base with `talosctl gen config`, pin the Kubernetes version one minor behind the newest, point the API endpoint at the virtual address on the server VLAN, and give etcd a partition on its own NVMe device — every etcd commit is an fsync, and nothing doing bulk writes shares that device.

02 Apply it to the three control-plane machines and run `talosctl bootstrap` on one. The installer wipes and repartitions the target disk, so confirm each machine's serial against the rack map first.

03 Join the rest. Thirteen become workers; the two spares take the same configuration and stay cordoned, so replacing a dead node is an uncordon rather than a build. At this size the control-plane nodes can stop taking workload — sixteen machines can spare three that only run etcd, and three that do nothing else are three that never get evicted at the wrong moment.

04 Install Cilium with `helm install` at the pinned chart version, kube-proxy replacement on, and the API host set to the virtual address, never a name: a name needs DNS, and DNS needs the network this step builds. Disable kube-proxy in the machine configuration in the same commit.

05 Verify with `kubectl` that no kube-proxy service chains survive on any node, then run the `cilium` connectivity test and read all of its output.

06 The acceptance test: pull the power on one machine at random, watch the API stay reachable and the pods reschedule, then power it back on and confirm it rejoins untouched. Do it while the cluster is empty; that is the last time it is cheap.

ROLLBACK

While the cluster is empty nothing here is hard to undo. Reverting the kube-proxy flag in the machine configuration and reapplying it takes a minute: an enormous blast radius, and a way back that is one flag. The point of no return is Move 12, where the first volume binds to a machine. Keep every configuration version in Git so a known-good one can be restored exactly.

BUILD · 0 MIN · HIGH RISK

Nothing yet — the managed cluster still carries production and stays until Move 20. Its per-hour control-plane charge is the first bill line to reach zero.

12 DISKS: WHAT GOES LOCAL, WHAT GOES ON CEPH

CUTOVER
0 MIN
0 60

LAYER	LEAVING	RISK	REVERSIBLE
BUILD	MANAGED BLOCK AND SHARED-FILE STORAGE	HIGH	IMMEDIATELY

Ceph takes three drives a node for anything that must survive a node dying. The fourth stays raw under PostgreSQL, which already replicates itself.

AWS	**EBS and EFS**	a gp3 volume lives in one Availability Zone and caps at 16,000 IOPS however large it grows, and EFS in bursting mode spends credits that run out.
GOOGLE	**Persistent Disk and Filestore**	Persistent Disk performance scales with provisioned size and the attached machine's vCPU count, and a Filestore instance grows but never shrinks.
AZURE	**Managed Disks and Azure Files**	Premium SSD steps in fixed tier sizes rather than a dial, and premium file shares need a FileStorage account that standard shares cannot move into.

BEFORE YOU START

ACCESS

- The device inventory from Move 09, with the four NVMe drives on each node known by serial
- A Talos machine configuration you can edit and roll, because disk layout is fixed at boot

SOFTWARE

- `helm` and `kubectl`, with the Rook chart pinned rather than tracking whatever is newest
- `ceph` for health, capacity and recovery output during the drive-pull test

PEOPLE

- One named owner for Ceph, who will read its health output on the day it is unhappy

WHY THIS WORKS

Two storage systems, and one rule for choosing between them. Ceph, run by Rook, takes anything that must survive a node dying without waking anybody: replicated block volumes, a shared filesystem for the few things that need one, and the object gateway Move 15 and Move 16 build on. PostgreSQL takes the other path, a raw local drive with nothing replicating beneath it. The volume it used to sit on provisioned a few thousand IOPS; the four drives already in the machine will do several hundred thousand. A database writes copies to its own standbys, so a layer repeating that work underneath spends flash to buy nothing.

SWAP

A workload that must move between nodes freely and cannot replicate itself goes on Ceph block, latency included. The choice is which layer pays for replication.

DO IT FASTER

Name the storage classes for the promise they make, not the technology beneath. An engineer picking one is choosing durability, not a product.

WATCH OUT

Ceph says loudly that it is unwell and quietly why. Learn to read `ceph health detail` on a calm afternoon rather than during an outage.

LEFTOVERS

The managed volumes, file shares and their snapshots bill right through Stage 4. This Move stands the replacement up; it stops nothing.

$9,400/mo	$850/mo	91%	0 min	6 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Nothing this week. The managed disks and shares carry production until Stage 4 moves what is on them; the line dies in Move 15 for the buckets and Move 16 for the database.

NEEDS 11 A cluster, in an afternoon

BUILD

12 DISKS: WHAT GOES LOCAL, WHAT GOES ON CEPH

6 STEPS · 0 MIN · HIGH RISK

BUILD

THE RUNBOOK

01 Name the owner for Ceph, then write the split into the Talos machine configuration: three of the four NVMe devices per node to Ceph, the fourth raw for PostgreSQL. Layout is fixed at boot, so changing it later means draining one node at a time.

02 Install Rook with `helm install rook-ceph rook-release/rook-ceph --version 1.16.9` and let the operator adopt the 15 devices offered. Take the Ceph release that chart version ships with, not the newest published.

03 Create a replicated block pool with three copies and `min_size 2`, a shared filesystem, and the object gateway on its own pool. Erasure coding is arguable at sixteen nodes and wrong at five, and this pool is block storage under a database: replication is the answer either way, and a pool holding data will not take a different scheme later.

04 Size for recovery, not for today. Losing one node of five pushes a fifth of the data onto the survivors, so run `ceph df` and hold the pools under 70 per cent. Fuller than that, recovery backfills into a wall and writes start failing.

05 Bind a claim with `kubectl`, write to it, reboot the node beneath it and check the contents survived. Repeat on the shared filesystem, then create the first bucket on the gateway; Move 15 and Move 16 expect that endpoint already.

06 Pull a drive from a running node and time the recovery with `ceph -s`, once the claims above are verified and nothing real is bound. Reseat it and watch the cluster take it back. That number is what somebody works against the first time a device dies for real.

ROLLBACK

Nothing is bound yet, so removing Rook and handing the devices back is a morning's work — until the first claim binds or the first object lands in the gateway. That is the point of no return: after it, leaving Ceph is a data migration with a write stop, not a configuration change. Prove the way back before you get there — snapshot a test volume, delete the claim, restore it, confirm the contents match. The raw drives hold nothing until Move 16.

BUILD · 0 MIN · HIGH RISK

Nothing this week. The managed disks and shares carry production until Stage 4 moves what is on them; the line dies in Move 15 for the buckets and Move 16 for the database.

STAGE 4 · IMAGES, THE FIRST SERVICE, THE BUCKETS, AND THE DATABASE

IV

MOVE

The application first, then the state. The stage that can lose a company its data, and therefore the one with the longest overlap windows, the most rehearsal and the fewest one-way steps.

4 MOVES	**3** ZERO DOWNTIME	**2** HIGH RISK	**15** MINUTES, TOTAL

IF YOU ONLY DO THREE

13

IMAGES, SECRETS AND ONE-COMMAND DEPLOYS

Three pieces of plumbing, then staging moves onto them: the first real workload on hardware you own, and the only one …

15

BUCKETS, CACHE AND QUEUES

Objects, cache and queues move by copying to a second place, then changing an endpoint. None of them needs writes …

14

THE FIRST SERVICE, END TO END

The cheapest stateless service goes first, running on your hardware against the cloud database across the link, with …

IMAGES, SECRETS AND ONE-COMMAND DEPLOYS · THE FIRST SERVICE, END TO END · BUCKETS, CACHE AND QUEUES · POSTGRES, THE ONE THAT MATTERS

2 MEDIUM RISK | 2 HIGH RISK

13 IMAGES, SECRETS AND ONE-COMMAND DEPLOYS

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
MOVE	MANAGED REGISTRIES, SECRET STORES AND HOSTED CI RUNNERS	MEDIUM	IMMEDIATELY

> Three pieces of plumbing, then staging moves onto them: the first real workload on hardware you own, and the only one whose bad afternoon costs you nothing.

AWS	**ECR and Secrets Manager**	the registry token expires every twelve hours, and a secret you remove holds its name for thirty days unless you ask for the seven-day minimum.
GOOGLE	**Artifact Registry and Secret Manager**	values are immutable, so every rotation leaves a billable version until destroyed.
AZURE	**Container Registry and Key Vault**	soft-delete cannot be switched off, and purge protection, once on, holds the name ninety days.

BEFORE YOU START

ACCESS

- A Git repository the cluster can read, with a deploy key held outside it
- An account on the registry you push to today, so the cache can pull private images

SOFTWARE

- Argo CD 3.0 and Harbor 2.12, both installed with `helm` at pinned chart versions
- `sops` and `age` for encrypting secrets, with the private key kept off the cluster
- A runner controller for your forge, ephemeral rather than long-lived

PEOPLE

- One engineer who owns the repository layout; two owners produce two conventions

WHY THIS WORKS

Nothing real has moved yet, so this is the moment for plumbing. A registry you own makes a build that cannot reach its base image a problem you fix, not an outage you watch. Cluster state in a repository, reconciled by Argo CD, makes every Move that follows a commit, and a commit can be reverted. Secrets encrypted there, under a key held outside the cluster, mean a cluster rebuilt from bare metal gets its credentials back. Runners come last because they hold most of the $640: hosted minutes bill by the minute, and a spiky build queue pays for waiting. Then staging goes, and it goes before anything a customer touches. This is the sequencing decision the whole stage rests on. Staging is a real workload — real images, real secrets, real deploys, real people who will tell you within the hour when it is wrong — and it is the only real workload whose bad afternoon appears on no invoice, breaches no agreement and wakes nobody. Every class of problem the platform has, it has under staging first: a storage class that does not bind, a network policy nobody wrote down, a base image that only existed in a cache, a deploy that works once and not twice. You would rather find those with your own engineers complaining in a channel than with Move 14's production service in front of users. It is also the honest rehearsal for the ones after it, because a staging estate is usually the same shapes as production at a tenth of the size.

SWAP

A plain registry over the same object store drops the database and cache Harbor needs, and with them projects, quotas and scanning.

DO IT FASTER

Do the cache in an afternoon and leave the registry of record a fortnight; most of the benefit arrives with the first half.

WATCH OUT

The registry has to run for the cluster to start, and the cluster for the registry to serve. Power the rack down and up once, deliberately, before an outage asks it.

WATCH OUT

Staging that still talks to production data stores has not moved, it has been relocated. Point it at its own copies before step seven.

LEFTOVERS

The cloud registry bills storage and egress until its images go, a Move 20 decision, and hosted runner minutes continue for anything still routed at them.

$2,600/mo	$260/mo	90%	0 min	7 days	2 weeks
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The hosted runner minutes, once a week of builds has passed on your own nodes with no fallback. The non-production estate's cloud compute, once the fortnight in step seven is done and nobody has asked to go back. The cloud registry and its secret store stay funded until Move 20.

NEEDS 11 A cluster, in an afternoon 12 Disks: what goes local, what goes on Ceph

THE RUNBOOK

01 Stand the registry up as a cache. `helm install` Harbor at chart version 1.16.2 on the Ceph object store from Move 12, proxying the registry you use today. Repoint pulls and confirm workloads still start.

02 Make it where images are pushed. Move the pipeline's push target across and mirror ninety days of tags, so both registries hold the same images while yours is on trial.

03 Put the cluster in the repository. `helm install` Argo CD at chart version 8.1.3, as an application of applications, so one path rebuilds namespaces, deployments, network policy and Move 12's storage classes. Bootstrap credentials come from outside the cluster.

04 Encrypt every secret into the same repository with `sops`, under an `age` key kept in a password manager and one offline copy. A rebuilt cluster is seeded with that key by hand; the key never lives inside the thing being rebuilt.

05 Delete a non-production namespace and let the reconciler put it back. Do this only after step four, and check the workload returns with its secrets attached.

06 Install the ephemeral runner controller at chart version 0.12.1 and route builds to it by label. Then the acceptance test: an engineer changes one line and the image builds, lands in your registry and deploys with nobody at a console.

07 Move the whole non-production estate — staging, review environments, whatever the branch builds deploy to — and leave it there. Point its DNS at the new cluster, let its data stay wherever it is for now, and tell the engineers who use it that it has moved and where to complain. Then use it: for a fortnight, every branch deploys here and nowhere else. The bugs this finds are the bugs Move 14 would otherwise find in front of users, and the cost of finding them here is somebody saying the preview is slow.

ROLLBACK

All of this is a repository: a bad change is a revert, and the reconciler catches up. Pulls point back at the cloud registry, builds back at hosted runners by one label. The point of no return is the day the cloud registry's images go, because a restore then means rebuilding from source, not pulling a tag. Keep the cloud secret store populated until a rebuilt cluster has decrypted its own secrets unaided.

MOVE · 0 MIN · MEDIUM RISK

The hosted runner minutes, once a week of builds has passed on your own nodes with no fallback. The non-production estate's cloud compute, once the fortnight in step seven is done and nobody has asked to go back. The cloud registry and its secret store stay funded until Move 20.

14 THE FIRST SERVICE, END TO END

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
MOVE	MANAGED CONTAINER COMPUTE	MEDIUM	IMMEDIATELY

> The cheapest stateless service goes first, running on your hardware against the cloud database across the link, with a DNS weight as the way back

AWS	**ECS on Fargate**	task credentials come from the metadata endpoint at 169.254.170.2, which does not exist on your nodes.
GOOGLE	**Cloud Run**	between requests the CPU is throttled to almost nothing unless it is always allocated, so idle timers stop firing.
AZURE	**Container Apps**	the outbound address belongs to the environment, not the app, so rebuilding it changes what partners must allowlist.

BEFORE YOU START

ACCESS

- Write access to the Argo CD repository holding the cluster's application manifests
- Permission to edit the weighted DNS records for the service's hostname

SOFTWARE

- `kubectl` and `argocd`, both already pointed at the new cluster
- `dig` for confirming what public resolvers return during the ramp

PEOPLE

- The service owner, told which week the ramp runs and what would stop it
- Whoever maintains egress allowlists on partner systems, warned in advance

WHY THIS WORKS

The first service to leave is chosen for how little it matters, because its job is to produce evidence rather than savings. Its database stays in the cloud, so the only new variables are the platform, the network path and the identity model. The link adds latency, and the number measured here decides which later services can live with a database on the far side of it and which must move in the same window as their data. The exit is a weighted DNS record: one value, one cache expiry. Nothing else returns as much for as little exposure.

SWAP

If the service sits behind a proxy you already control, shift weight there rather than in DNS; the abort is then immediate, not TTL-bound.

DO IT FASTER

Ramp on a Tuesday morning with the owner beside you. Two people reading one graph is most of what the first one is for. After that the rate is set by how many teams can watch their own service go, not by the platform: five a week is a team a day, and a smaller estate should run slower rather than pretending the graph reads itself.

WATCH OUT

A mean response time stays flat while one pod in four times out. Alert on percentiles and status codes, not averages.

LEFTOVERS

The old record sits at zero weight and the allowlists carry both ranges. Both are free and both get forgotten, so date each.

$30,980/mo	$0/mo	100%	0 min	20 days	8 weeks
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed container service, its task revisions and its log retention — but not after the first service's week at full weight, which is the mistake this line used to invite. It goes when the LAST service off step 6 has held for a week, which is a week after the eight-week cadence ends rather than a week after it starts.

NEEDS 13 Images, secrets and one-command deploys

MOVE

14 THE FIRST SERVICE, END TO END

6 STEPS · 0 MIN · MEDIUM RISK

THE RUNBOOK

01 Choose the cheapest stateless service, not the most interesting one. The right candidate is one whose failure is a shrug and whose owner will not escalate over a slow hour.

02 Commit its manifests to the Argo CD repository and let `argocd` sync them. Keep the connection string on the cloud database, swap anything reading instance metadata for a real credential, and tell the partners who filter on egress addresses.

03 With no live traffic, drive synthetic requests at the new copy and compare latency percentiles against the cloud copy. Write the added milliseconds down.

04 Ramp the weighted record: one per cent, ten, fifty, all of it. Hold each step a business day, confirm the split with `dig`, and stop on any rise in error rate or the ninety-ninth percentile.

05 Soak a week at full weight, watching node disk with `kubectl`: scratch files that were ephemeral on the managed runtime still are, but now they fill a machine you own.

06 Then repeat across the stateless estate, five services a week, batched by the team that owns them so one person is not the queue. After the third, nobody asks for a plan. This is most of the Move: the first service is five days and the forty-odd behind it are about a third of a day each, which is what the strip books as effort. The eight weeks of cadence are not booked twice — they are the calendar those days are spread across, not a window in which nobody works.

MOVE

ROLLBACK

Every step before the ramp is additive: two copies exist and the cloud copy carries the traffic. Backing out is setting the weighted record to zero and waiting one TTL. There is no point of no return here — no state has moved, so there is nothing to restore. The exception is a partner's allowlist entry: leave the old address in place for the week.

MOVE · 0 MIN · MEDIUM RISK

The managed container service, its task revisions and its log retention — but not after the first service's week at full weight, which is the mistake this line used to invite. It goes when the LAST service off step 6 has held for a week, which is a week after the eight-week cadence ends rather than a week after it starts.

15 BUCKETS, CACHE AND QUEUES

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
MOVE	MANAGED OBJECT STORAGE, REDIS AND QUEUES	HIGH	30 DAYS

> Objects, cache and queues move by copying to a second place, then changing an endpoint. None of them needs writes stopped.

AWS	**S3, ElastiCache and SQS**	SQS now carries a megabyte a message, so a large payload may be sitting inline rather than as a pointer into a bucket.
GOOGLE	**Cloud Storage, Memorystore and Pub/Sub**	Pub/Sub acknowledges per subscription, so two subscriptions deliver everything twice.
AZURE	**Blob Storage, Azure Cache for Redis and Service Bus**	sessions and duplicate detection begin at the Standard tier; Basic has no topics.

MOVE

BEFORE YOU START

ACCESS

- Read access to every bucket and the managed cache, and write on the Ceph gateway (Move 12)
- The queue inventory: visibility timeouts, dead-letter rules and message size ceilings

SOFTWARE

- `rclone` 1.68 or newer, comparing checksums rather than size and time
- Valkey 8.0 with `valkey-cli`, and NATS JetStream 2.10 with the `nats` command line

PEOPLE

- The owner of each key pattern and each queue consumer: both switches are code changes

WHY THIS WORKS

The database is the hard migration; this one runs beside it. Objects, cache and queues can each be written to two places at once, so the work is a copy repeated until the delta is boring, then a configuration change. Writes never stop. The risk is not the bytes but the assumptions in the code: an object store that promised more than the standard did, a cache holding sessions nobody labelled as state, a queue whose visibility timeout the consumer extends. Reading source finds those. Watching a transfer does not.

SWAP

One small bucket read by one service needs no repeated passes. Copy it once, verify it once.

DO IT FASTER

Start the bulk copy the week the cluster comes up. It is bandwidth, not attention, and the last delta takes an hour.

WATCH OUT

Presigned links issued before the endpoint changed resolve to the cloud until expiry; somebody always set one to a year.

LEFTOVERS

Non-current object versions bill on after the live ones go, and cache nodes bill until the last client moves.

$12,800/mo	$1,150/mo	91%	0 min	8 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The cache nodes and the managed queues, after thirty days and a clean billing cycle. The object line shrinks rather than going: the archive stays.

NEEDS 12 Disks: what goes local, what goes on Ceph 14 The first service, end to end

15 BUCKETS, CACHE AND QUEUES

6 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Sort the object estate by size and last access. Hot buckets move; archive stays, because egress on extracting terabytes exceeds years of its storage fee.

02 Copy the hot buckets with `rclone` onto the Ceph gateway, repeating until a pass takes minutes. A multipart entity tag is a digest of digests: it compares only if part sizes match.

03 Change the endpoint, then run a last `rclone` pass for what was written during the swap. Read the client code for listing order, read-after-write on overwrites, part size.

04 Scan the cache key space by pattern. Patterns with no expiry are sessions, counters and locks — state, not cache — and wait for Move 16. The owner confirms; the rest point at Valkey 8.0, checked with `valkey-cli`, and take a cold start in a quiet hour.

05 Stand up NATS JetStream 2.10 beside the old queues. Make consumers idempotent, then have producers write to both for a day, checking each visibility timeout against a long job.

06 Drain the old queues to zero, watch `nats` take the load, then stop the old consumers. Both copies are retained for thirty days; nothing is deleted.

MOVE

ROLLBACK

Nothing here is deleted, so backing out is three configuration changes: object endpoint, cache endpoint, broker. Objects written to Ceph afterwards are restored by copying the other way. The point of no return is the first message acknowledged only on the new broker.

MOVE · 0 MIN · HIGH RISK

The cache nodes and the managed queues, after thirty days and a clean billing cycle. The object line shrinks rather than going: the archive stays.

16 POSTGRES, THE ONE THAT MATTERS

CUTOVER
15 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
MOVE	**MANAGED POSTGRESQL**	HIGH	7 DAYS

> Fifteen minutes of stopped writes, at the end of a week in which the new cluster was caught up, backed up, and restored once on purpose.

AWS	**RDS for PostgreSQL**	the default parameter group cannot be edited, so `rds.logical_replication` needs a custom group and a reboot.
GOOGLE	**Cloud SQL for PostgreSQL**	storage auto-grows to hold retained write-ahead log and never shrinks, so it bills at the high-water mark.
AZURE	**Database for PostgreSQL flexible server**	`CREATE EXTENSION` fails until the name is in the `azure.extensions` allowlist, and anything wanting shared memory needs `shared_preload_libraries` too, which does need a restart.

BEFORE YOU START

ACCESS

- Logical decoding enabled on the managed instance, with the reboot taken
- Credentials for a bucket on the object gateway from Move 12, separate from the cluster's

SOFTWARE

- CloudNativePG 1.25 or newer, failed over by hand at least once already
- The Barman Cloud plugin for CloudNativePG, pinned, because the operator writes `archive_command` itself and will not take one from you
- `psql` and `pg_dump` no older than the managed server

PEOPLE

- The application owner, present for the window when writes stop

WHY THIS WORKS

Nothing here is new except the data. The local NVMe and the object gateway the backups go to both came up in Move 12, that gateway has carried the application's own buckets since Move 15, and the application has run on the cluster since Move 14. CloudNativePG supplies the part nobody wants to write: a primary, two standbys, and an operator that promotes one without a human awake at three in the morning. The copy is a stream, not a dump, so the new cluster is caught up and dull for days before anyone books a window. The rehearsed restore is what makes the fifteen minutes safe.

MOVE

SWAP

Leave the managed instance subscribed to the new cluster for the week, replicating backwards. One more week of the old bill turns rollback into a repoint.

DO IT FASTER

Split the publication into several subscriptions grouped by table size, so the initial copy runs in parallel, not table by table.

WATCH OUT

Replication carries rows only. Sequences, large objects, schema changes and tuned parameters stay behind, and a sequence left at one collides on the first insert.

LEFTOVERS

The replication slot on the source outlives its subscription and retains write-ahead log for a reader that has gone.

$19,500/mo	$0/mo	100%	15 min	12 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The managed instance, its read replicas and their snapshots, after the seven days and once an invoice shows the line gone.

NEEDS 12 Disks: what goes local, what goes on Ceph 13 Images, secrets and one-command deploys 15 Buckets, cache and queues

16 POSTGRES, THE ONE THAT MATTERS

THE RUNBOOK

01 List the extensions and roles on the managed instance with `psql`. What exists only on the provider's fork is replaced, re-expressed, or blocks the Move.

02 Copy the schema with `pg_dump --schema-only` onto a CloudNativePG cluster of one primary and two standbys on local NVMe, deferring the biggest indexes.

03 Create the logical replication publication and subscription and let it run for days. Read `pg_stat_subscription` each morning: a stalled subscriber fills the source's disk.

04 Declare the object store as the cluster's backup destination so the Barman Cloud plugin drives it, turn on continuous archiving, watch the first write-ahead log segments arrive in the bucket, and only then take the full backup.

05 Restore that backup into a scratch cluster, replay to a chosen timestamp, and count rows against production. Time it. The cutover waits until this passes.

06 Open the window. Stop writes at the application, wait for lag to reach zero, advance every sequence, promote, repoint, start writes.

MOVE

ROLLBACK

Until writes resume, backing out is releasing the pause. The point of no return is the first commit on the new cluster: after that the two have diverged, and going back means replaying those writes or losing them. The managed instance stays up with its backups on for seven days, a copy that can still be restored to a timestamp.

MOVE · 15 MIN · HIGH RISK

The managed instance, its read replicas and their snapshots, after the seven days and once an invoice shows the line gone.

STAGE 5 · THE EDGE, GO-LIVE, BACKUPS, THE PAGER AND THE LAST ACCOUNT

V

RUN

The traffic, the pager and the last account. Every article about leaving the cloud stops before this stage, and every migration that regrets itself failed inside it.

4	3	3	10
MOVES	ZERO DOWNTIME	HIGH RISK	MINUTES, TOTAL

IF YOU ONLY DO THREE

17

THE FRONT DOOR

The new edge goes up beside the one carrying users, with its own address and certificates, proved from a host file …

19

BACKUPS YOU HAVE RESTORED, AND THE PAGER

Velero, a continuous database archive and a copy somewhere you are not leaving — then the drill that turns a guessed …

18

GO-LIVE, AND HOW YOU ABORT

The weight moves in four steps and comes back in one, which is why the time-to-live drops a day early and the old …

THE FRONT DOOR · GO-LIVE, AND HOW YOU ABORT · BACKUPS YOU HAVE RESTORED, AND THE PAGER · CLOSING THE ACCOUNT

1 MEDIUM RISK | 3 HIGH RISK

17 THE FRONT DOOR

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
RUN	MANAGED LAYER-7 BALANCERS AND CERTIFICATE SERVICES	MEDIUM	IMMEDIATELY

> The new edge goes up beside the one carrying users, with its own address and certificates, proved from a host file entry while real traffic goes elsewhere.

AWS	**Application Load Balancer and ACM**	the validation record must stay in the zone forever, because every renewal re-checks it.
GOOGLE	**external Application Load Balancer and Certificate Manager**	URL map, target proxy and forwarding rule are separate objects, and the reserved address outlives all three.
AZURE	**Application Gateway and Key Vault certificates**	the gateway reads its certificate through a managed identity, and withdrawing that grant fails the gateway.

BEFORE YOU START

ACCESS

- The public address range from Move 08, with one address held back for the gateway
- Write access to the repository from Move 13, the one Argo CD reconciles
- An export of every listener rule, redirect, rewrite and health check from the managed balancer

SOFTWARE

- `helm`, with a pinned chart version for each of the two installs below
- `cmctl` and `openssl`, to check issuance and to read what a listener actually serves

PEOPLE

- Whoever owns the two most valuable transactions, to walk them end to end first

WHY THIS WORKS

An edge cannot be half-tested once it is live, so this one is built beside the live one. Envoy Gateway takes an address from the range ordered in Move 08, terminates its own certificates, and routes to the services Move 14 and Move 16 already put on your hardware. No user arrives, because no DNS record points at it. That exposes the hard part cheaply: the listener rules and rewrites accumulated in the managed balancer over three years, which nobody has a full list of. As Gateway API objects they can be read and diffed.

RUN

SWAP

Under a dozen rules, skip the table and write the routes directly. It earns its keep around thirty.

DO IT FASTER

Stand the gateway up against one low-traffic service and keep that route as a permanent smoke test. It exercises address, certificate and route before the table exists.

WATCH OUT

A renewed certificate and a reloaded one are different events. Envoy takes a new secret in seconds; a legacy proxy behind it may not. cert-manager renews at day sixty of a ninety-day certificate, so the stale copy stays valid for another thirty and the symptom arrives on day ninety.

LEFTOVERS

The managed balancer charges per hour and per capacity unit until Move 18, and its validation records must stay in the zone or its certificates stop renewing.

$2,400/mo	$320/mo	87%	0 min	4 days	—
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Nothing yet. The managed balancer, its listener rules and its certificate service switch off in Move 18, where the $780 line stops.

NEEDS 14 The first service, end to end 16 Postgres, the one that matters

17 THE FRONT DOOR

THE RUNBOOK

01 Install both controllers with `helm` at fixed versions — `helm install eg oci://docker.io/envoyproxy/gateway-helm --version v1.4.2`, and cert-manager at `v1.17.2`. Commit the values to the repository so Argo CD owns them.

02 Bind the gateway to one address from the Move 08 range as a virtual IP in layer-2 mode. Cut power to the node holding it and time the recovery: on one rack that is neighbour cache expiry, not anycast, and ten to thirty seconds is honest.

03 Issue certificates through cert-manager, confirm with `cmctl status certificate`, then read what the listener serves using `openssl s_client -connect`. Prove renewal in a scratch cluster by winding its clock past the threshold, not by waiting until day eighty-nine.

04 Read the balancer export line by line into a table: one row per listener rule, path condition, redirect, header rewrite, sticky-session setting and health check. The rule nobody remembers is the one a customer depends on.

05 Translate each row into an HTTPRoute in the repository and write one request that proves it. Priority ordering and specificity ordering disagree, so a rule shadowed for two years comes back to life once translated faithfully.

06 Point a laptop at the new address with a host file entry and walk sign-in, the two most valuable transactions, a large upload and any websocket. Cut nothing over; Move 18 moves the traffic.

RUN

ROLLBACK

Nothing here serves users, so backing out is removing the gateway objects and the address assignment, once no host file entry still points at them. There is no point of no return in this Move, by design. Keep the definitions in the repository so the edge can be restored in an afternoon.

RUN · 0 MIN · MEDIUM RISK

Nothing yet. The managed balancer, its listener rules and its certificate service switch off in Move 18, where the $780 line stops.

18 GO-LIVE, AND HOW YOU ABORT

CUTOVER
10 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
RUN	WEIGHTED DNS AND THE PARALLEL MANAGED EDGE	HIGH	7 DAYS

> The weight moves in four steps and comes back in one, which is why the time-to-live drops a day early and the old edge stays up a week.

AWS	**Route 53 weighted records**	a group weighted entirely to zero answers with every record in it, so drain by raising the survivor.
GOOGLE	**Cloud DNS routing policies**	a policy cannot coexist with a plain record of the same name and type, so creating one replaces that set.
AZURE	**Azure DNS and Traffic Manager**	weighting lives in the profile, not the zone, so the apex needs an alias record and a second time-to-live.

BEFORE YOU START

ACCESS

- Write access to the authoritative zone, and a second person who has it too
- The change window agreed with whoever answers the support line

SOFTWARE

- `dig` run from two networks you do not control, and a resolver you have never used
- `argocd` sync windows, or another way to hold every deployment for the day

PEOPLE

- One named decider, and the abort criteria they read out rather than argue

WHY THIS WORKS

Almost all of this Move is preparation, and the preparation is writing things down while nobody is under pressure. The mechanism is one field: a weight on a record, moved four times and movable back once. It is safe because the destination already carries real traffic and already holds the same rows as the source, so nothing is proved here for the first time. The time-to-live decides how long an abort takes, so it drops ahead of the day and is checked rather than assumed. Each soak spans a traffic cycle, because a filling connection table hides in a quiet minute.

SWAP

Where a contract forbids a freeze, run the ramp in the customer's quiet hours in smaller increments across a fortnight.

DO IT FASTER

Take the one per cent step a week early, on its own. It finds the certificate and header faults while there is time to fix them.

WATCH OUT

Some corporate resolvers and mobile stacks ignore the time-to-live outright and reach the old address for days. Hence the week.

LEFTOVERS

The lowered time-to-live is itself a leftover, multiplying query volume and query billing on a zone nobody reads.

$8,900/mo	$0/mo	100%	10 min	3 days	1 week
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The weighted record and the managed edge behind it, seven days after full weight, and with them the egress line that was most of the $8,900.

NEEDS 17 The front door

RUN

THE RUNBOOK

01 Lower the record's time-to-live to 60 in the authoritative zone a day ahead, then confirm it with `dig` on several resolvers, not the registrar's view.

02 Write the abort criteria as numbers: error rate above a figure, tail latency above a figure, any data-integrity report at all. Name the decider on it.

03 Tell the clients a record cannot move: mobile builds with a pinned host, batch jobs holding an old address, third parties allowlisting one.

04 Open the change window and freeze deploys with an `argocd` sync window over it. A release landing mid-shift cannot be told apart from the shift.

05 Move the weight to one per cent and soak. Then ten, then fifty, then all of it. The ten minutes in the strip is the last step, not the day.

06 Keep the managed edge configured, resolvable and billed at full weight for seven days. Nothing is deleted before the week is out, so the abort stays a field.

RUN

ROLLBACK

Putting the weight back returns new connections to the managed edge inside one time-to-live, which is why it was lowered first. That is the whole procedure for seven days. The point of no return is not the weight change but switching the managed edge off when the week is out. If the data failed rather than the traffic, no weight helps and the way back is the database restore.

RUN · 10 MIN · HIGH RISK

The weighted record and the managed edge behind it, seven days after full weight, and with them the egress line that was most of the $8,900.

19 BACKUPS YOU HAVE RESTORED, AND THE PAGER

CUTOVER
0 MIN

0 · 60

LAYER	LEAVING	RISK	REVERSIBLE
RUN	MANAGED BACKUP, ALARMS AND A RENTED PAGING SERVICE	HIGH	IMMEDIATELY

> Velero, a continuous database archive and a copy somewhere you are not leaving — then the drill that turns a guessed recovery time into a measured one.

AWS	**AWS Backup and CloudWatch alarms**	a vault under compliance-mode lock cannot be shortened or removed until its retention expires.
GOOGLE	**Backup and DR Service and Cloud Monitoring**	a backup vault's enforced retention makes a backup indelible until it expires, by you or by Google, so the copy outlives the decision to leave.
AZURE	**Azure Backup and Azure Monitor alerts**	a Recovery Services vault protects only resources in its own region, and soft delete holds removed items for fourteen days.

BEFORE YOU START

ACCESS

- An account at an object storage provider unrelated to the cloud you are closing, with object lock
- Five days of one engineer's time for the rebuild drill, on a date nobody may move

SOFTWARE

- `velero` 1.15 and the Barman Cloud plugin from Move 16, both pinned in the repository from Move 13
- `rclone` 1.68 for the copy out to the third provider
- Prometheus and Loki with a series budget agreed before the first scrape

PEOPLE

- The existing operations rota, a named second escalation contact and a rule about what waits until morning
- The facility or rental provider's physical-support escalation and approved intervention runbooks

WHY THIS WORKS

Two failures end a company here: the data cannot be got back, and nobody was woken. Schedules are cheap and worthless until somebody has rebuilt from them, so the deliverable is the drill — an estate reconstructed from the repository and the object store while production carries on untouched, with a stopwatch running. That figure is the recovery time objective, and it is usually several times the number in the plan. Monitoring runs on your platform; the alerting runs outside it, because a failed cluster cannot report its own failure. The existing cloud operations rota transitions with the services. Physical faults go to the contracted remote hands team or rental provider; your engineers retain software recovery and incident command within the same recurring-hours budget, validated during the pilot.

SWAP

If the drill on the spare machine is disruptive, rent a dedicated server by the month and rebuild into that instead.

DO IT FASTER

Make the drill a script, not a document. The second run then takes an afternoon rather than a week.

WATCH OUT

Cluster objects without volume data restore an estate that starts cleanly and holds nothing. Count rows before calling the drill a pass.

LEFTOVERS

Backup vaults bill after the resources they protected are gone, and one under a retention lock stays until it expires.

$4,800/mo	$1,400/mo	71%	0 min	5 days	–
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF Managed backup vaults, their cross-region copies, the alarms and their ingestion, and the rented pager seats — once a restore of your own has been timed and one real page has woken somebody.

NEEDS 16 Postgres, the one that matters 18 Go-live, and how you abort

19 BACKUPS YOU HAVE RESTORED, AND THE PAGER

6 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Schedule `velero` nightly for cluster objects, and let the Barman Cloud plugin archive the database from Move 16 continuously, both into the Ceph object gateway built in Move 12.

02 Copy that store out with `rclone` to a provider that is neither the rack nor the account being closed, with object lock on and append-only credentials, so one identity cannot reach both copies.

03 Do the drill. On the spare machine from Move 06, from the Git repository and those backups alone, build a cluster, restore the database and bring one service up, with nothing touching the original. Time it: that number is the recovery time objective, and the plan that disagrees is wrong.

04 Stand Prometheus and Loki up on your own nodes against the series budget, with drop rules at scrape time. A monitoring bill that outgrows the estate it watches is a cloud habit that follows you home.

05 Put the alerting where the cluster cannot take it down. OneUptime runs the probes, the rota, the escalation and the status page from outside; the author of this book founded OneUptime, and the copyright page says so. Add a dead-man's-switch heartbeat.

06 Transition the existing cloud rota to the new platform. Route physical faults to remote hands or the rental provider under the support contract; keep software recovery with the operations team. Test both escalation paths, name a backup contact, record what waits until morning and measure the hours against Move 03's budget.

RUN

ROLLBACK

Nothing here alters what serves traffic, so backing out is removing two schedules and leaving the managed alarms enabled. The point of no return is the day the provider's vaults age out: after that the only copy of last month is the one you wrote. Keep them until a restore from your own has been timed.

RUN · 0 MIN · HIGH RISK

Managed backup vaults, their cross-region copies, the alarms and their ingestion, and the rented pager seats — once a restore of your own has been timed and one real page has woken somebody.

20 CLOSING THE ACCOUNT

CUTOVER
0 MIN

0 60

LAYER	LEAVING	RISK	REVERSIBLE
RUN	THE PROVIDER ORGANISATION, ITS AUDIT TRAIL AND ITS SUPPORT PLAN	HIGH	30 DAYS

> The estate moved a month ago. The account did not: a dormant organisation still bills, still holds credentials, and still lets someone start an instance in it.

AWS	**the management account and its support plan**	closing a member account takes the management account, but making one standalone first needs its own contact details, payment method and support plan.
GOOGLE	**organisation and its projects**	project deletion is scheduled rather than immediate, and a lien refuses the request outright.
AZURE	**subscriptions and the Entra tenant**	the tenant refuses deletion while any subscription or application registration still exists.

BEFORE YOU START

ACCESS

- Root credentials for the management account, held by two named people
- The registrar login, often held by one person who has since left

SOFTWARE

- `aws`, `gcloud` or `az` signed in with an identity that can read billing
- `rclone` 1.68 or newer to copy the audit trail and billing history somewhere you own

PEOPLE

- Whoever answers the auditor, because the attestations you inherited become yours to produce
- The finance owner who has to see the invoice reach zero and say so

WHY THIS WORKS

A migration that stops at the traffic cutover leaves about $1,240 a month and a quieter second problem: live credentials, an audit surface someone will still ask about, and enough idle capacity to make drifting back the cheapest decision in the room. Nothing here is deleted until a full billing month has passed with the line at zero and no ticket filed. The failures that follow a migration are monthly rather than daily — the job that runs on the twenty-eighth and cannot find its bucket. Switching off and closing down stay four weeks apart, with everything irreversible on the far side of that gap.

RUN

SWAP

Where one thing genuinely has to stay — an archive whose egress you will not pay — move it into a small account of its own and close everything around it.

DO IT FASTER

Begin the key management waiting periods on the first day of the wait, not the last. Seven to thirty days is the AWS range; Google's can be set as long as a hundred and twenty, and a Key Vault holds keys for a retention period fixed when the vault was created. Nothing else waits on any of them.

WATCH OUT

Committed spend does not end when usage does. A Savings Plans commitment keeps billing against the organisation after every resource inside it has gone.

LEFTOVERS

Billing exports, budget alerts and marketplace subscriptions outlive what they were attached to. So does the support plan, which renews on its own anniversary rather than yours.

$3,200/mo	$0/mo	100%	0 min	4 days	4 weeks
WAS	NOW	SAVED	CUTOVER	EFFORT	WAIT

TURN OFF The account itself, and with it the support plan, the idle gateways, the log ingestion and the snapshot retention still billing a month after the last user request left. What is left is a platform rather than a project.

NEEDS 18 Go-live, and how you abort 19 Backups you have restored, and the pager

20 CLOSING THE ACCOUNT

6 STEPS · 0 MIN · HIGH RISK

THE RUNBOOK

01 Take the final invoice apart with `aws`, `gcloud` or `az`. Expect address translation gateways nothing routes through, CloudWatch log groups still ingesting, snapshots on a retention policy nobody reviews, a support plan priced against spend that has gone, and two subscriptions from a proof of concept three years ago.

02 Extract what cannot be recovered later: the audit trail for the period your auditor requires, the billing history, and the compliance attestations you inherited. Copy them out with `rclone`, then open one of each and check it reads.

03 Revisit the archive you decided in Move 15 to leave behind. Data parked in an account you are closing is not archived anywhere. Move it out, or agree in writing to delete it once someone has confirmed nothing still reads from it.

04 Move the domain before anything touches identity. Where the registrar sits with the provider — Route 53 is its own, and Azure fronts a partner under its own name — transfer it away, then put the lock back on.

05 Turn everything off and remove nothing. Stop the instances, drop the support plan to the free tier, keep the snapshots, and wait one full billing month with the compute lines at zero and no ticket filed. The snapshots and the volumes under the stopped instances go on billing, and are meant to: that residue is what keeps the month reversible.

06 Close it from the leaves inward. Federation and human users first, then access keys and roles, then the keys in Secrets Manager or Key Vault, because those decrypt archives you may still hold. Member accounts and subscriptions next; the organisation itself last.

RUN

ROLLBACK

Until the closure request this is ordinary work: an account switched off can be switched back on in an afternoon, which is why the wait is four weeks and not four days. The point of no return is the request itself. AWS will reopen a closed account for ninety days, a cancelled Azure subscription keeps its data for thirty to ninety, and a Google project stays scheduled for deletion for thirty — so a restore of anything left inside is possible for thirty days on all three and impossible after ninety. Use the wait to confirm the audit trail and billing history read correctly from your own copies.

RUN · 0 MIN · HIGH RISK

The account itself, and with it the support plan, the idle gateways, the log ingestion and the snapshot retention still billing a month after the last user request left. What is left is a platform rather than a project.

AFTERWARDS

A FEW THINGS WORTH KNOWING

None of these is a Move. They are the ten things that make the other 20 work, and most of them are the things people learn in the second year rather than the first.

THE BILL LAGS THE CHANGE BY A MONTH

A resource stopped on the third still bills for the first two days, and a reservation you no longer use bills for a year. Judge a Move by the invoice after next, never by the console.

CAPACITY YOU OWN IS CAPACITY YOU HAVE

The cloud taught a generation to treat headroom as waste. On your own metal, idle capacity is the thing that absorbs a bad deploy at four in the afternoon. Buy thirty per cent more than the model says.

TWO PEOPLE KNOW, OR NOBODY DOES

The single most common failure mode of a self-run platform is not hardware. It is one engineer who understands the network, and a holiday.

HARDWARE FAILS PREDICTABLY AND BORINGLY

Disks, fans, PSUs, optics, in that order, at a rate you can put in a spreadsheet. It is not the drama people expect. The drama is in the change you made on Friday.

KEEP ONE FOOT IN THE CLOUD ON PURPOSE

An account with a working credential, a Terraform state and a warm standby is not a failure of nerve. It is the cheapest disaster recovery site you will ever have.

THE SAVING IS REAL; THE FREEDOM IS THE POINT

Most teams start this for the bill and finish it for the other thing: being able to answer a question about your own system without opening a support case.

WRITE THE RUNBOOK BEFORE YOU NEED IT

Every Move in this book is a runbook because that is the artefact that survives the person who wrote it. A migration that leaves no runbooks behind has to be done again by whoever comes next.

YOU CAN GO BACK

The industry talks about repatriation as though it were one-way. It is not. 0 of the 20 Moves here are genuinely irreversible and the other 20 are not, and knowing which is which is most of the skill.

The best infrastructure decision you make this year will probably be unglamorous, take a fortnight, save a five-figure sum and never be written up anywhere. That is rather the point.

THE BOOK

20 Moves across five stages. 18 cost no downtime at all, 0 cannot be undone, and the whole book executed end to end costs 25 minutes of user-visible outage.

THE TYPE

Set in Archivo, drawn by Omnibus-Type, and JetBrains Mono, drawn by Philipp Nurullin and Konstantin Bulenkov. Both are open source. Every number in this book is set in the mono.

THE NUMBERS

Every price is a public list rate observed while writing, before any discount, and every saving is illustrative of a shape rather than a quotation. Check the arithmetic against your own invoice.

Disclosure: the author founded OneUptime, which this book recommends for alerting, on-call, incidents and status pages. It is recommended because it is Apache-2.0 licensed, self-hostable, and does in one place what that Move would otherwise ask you to assemble from four separate tools. The alternatives named beside it are real ones, and a reader who prefers them loses nothing else in this book.

HackerBay.io · Version 6.3.1 · The Moves and the typesetter that made this book are open source at github.com/OneUptime/back-to-metal